Year: 2026-2027 **Sem V-NEP**

Course: Agile Software Development & DevOps Lab

Class: T1 **Date of Submission: 11/10/2026**

A Mini Project Report on

Hostel Management System

A Room Allocation and Occupancy Platform Delivered Through an Agile and DevOps Pipeline

Submitted in partial fulfilment of the requirements of the course

Agile Software Development & DevOps Lab

Semester V (NEP) · Academic Year 2026–2027 · Class T1

Submitted by

Name	Roll No.
Soham Ajwani	2401004
Ananya Amesur	2401005
Janvi Chattani	2401018

Under the guidance of

Dr. Sanober Shaikh

Department of Artificial Intelligence & Data Science

Thadomal Shahani Engineering College

2026–2027

CERTIFICATE

This is to certify that the mini project entitled "Hostel Management System: A Room Allocation and Occupancy Platform Delivered Through an Agile and DevOps Pipeline" is a bona fide work of **Soham Ajwani (2401004)**, **Ananya Amesur (2401005)** and **Janvi Chattani (2401018)**, submitted in partial fulfilment of the requirements of the course Agile Software Development & DevOps Lab, Semester V, at Thadomal Shahani Engineering College during the academic year 2026–2027.

_____ _____

Project Guide Head of Department

Dr. Sanober Shaikh

Date: _____ Place: _____

ABSTRACT

Hostel allotment in many colleges is still handled through registers and spreadsheets. Wardens cannot see at a glance which rooms still have a free bed, a room under repair can be allotted by mistake, and there is no single record of which student sits in which room. The Hostel Management System addresses this with a web application in which a warden registers a student, picks only from rooms that are active and have a vacant bed, and sees the occupancy figures, room inventory and student registry update at once on a dark-themed dashboard.

The application was used as the vehicle for applying all seven experiments of the Agile Software Development & DevOps Lab to a single product. Work was planned in Jira as three epics and six stories (19 story points) in one sprint, HM Sprint 1, and all nine work items were closed. Each team member developed on a feature branch of a shared GitHub repository, and the frontend and DevOps branches reached `main` through pull requests #1 and #2. The backend, a FastAPI service backed by a Supabase cloud database, and the frontend, static pages served by Nginx, are packaged as Docker images and run together by Docker Compose on a private bridge network. A Jenkins pipeline checks out `main`, runs the pytest suite, records the results in JUnit format, verifies the Docker build, deploys the services and finishes with a smoke test of the health endpoint. Nagios Core then checks the frontend, the backend and its database connection, the Nginx proxy path and the disk space of the host.

The delivered system comprises 11 rooms with 22 beds, 12 of which were occupied at the time of capture (an occupancy rate of 54.5 %), eleven automated tests that all passed in 1.62 seconds, a Jenkins run that ended in SUCCESS with the health probe reporting the database as CONNECTED, and four Nagios service checks that were all in the OK state. The report documents each stage with the team’s own screenshots and explains how code that is committed to Git travels, without manual steps in between stages, to a running and monitored deployment.

Keywords: Agile, Jira, Git, Docker, Docker Compose, Jenkins, CI/CD, Nagios, FastAPI, Nginx, Supabase

CONTENTS

Chapter	Title
1	Introduction
2	Requirements and Agile Planning (Experiment 6)
3	System Design
4	Application Implementation
5	DevOps Implementation (Experiments 1–7)
6	Testing
7	Results and Discussion
8	Conclusion and Future Scope
	References
A	Appendix: Running the Project

1. Introduction

1.1 Background

A college hostel has a fixed number of rooms, each with a fixed number of beds, and every new admission has to be matched with a bed that is free and fit to be occupied. When this is done on paper or in a shared spreadsheet, the warden has to count occupants by hand to know which rooms are full, a room that is closed for maintenance can be allotted by mistake, and there is no quick way to answer simple questions such as how many vacancies are left in Block C or where a particular student was placed.

The Agile Software Development & DevOps Lab introduced, through seven separate experiments, the practices used to deliver software of this kind reliably: version control, containers, multi-container orchestration, continuous integration, automated deployment, Agile project tracking and monitoring. Each experiment was first performed on a small exercise. This mini project applies all seven to one realistic product, a hostel room-allocation platform, so that the way the tools work together becomes visible instead of each tool being seen on its own.

1.2 Problem Statement

To design, build and operate a web application that lets a warden register students and allocate them to rooms with live validation and occupancy statistics, and to deliver it through an Agile, automated pipeline in which every piece of work is tracked in Jira, versioned in Git, tested and deployed by Jenkins, packaged as Docker containers and observed by Nagios.

1.3 Objectives

1. Let the warden see all rooms with block, type, capacity, occupancy, monthly fee and status in one searchable and filterable table.
2. Register a student and assign a room in one step, offering only rooms that are active and have an open bed, and rejecting invalid input.
3. Show live key figures: total rooms, occupied beds, available vacancies and occupancy rate.
4. Plan the work in Jira with epics, story points, a sprint board and a completed-sprint report.
5. Keep all source code in a GitHub repository, with each team member working on a branch and merging through pull requests.
6. Containerise the backend and the frontend and run them together with a single Docker Compose command.
7. Build a Jenkins pipeline that checks out the code, runs the automated tests, verifies the Docker build, deploys the services and smoke-tests the result.
8. Monitor the availability of the frontend, the backend, the database connection and the host disk with Nagios.

1.4 Scope

The system targets one hostel made up of four blocks (A to D) with eleven rooms and twenty-two beds, as loaded in the demonstration data. Collection of fees, student login and role-based

access are outside the scope of this version; the monthly fee is displayed for information. The deployment target is a single Windows machine running Docker Desktop. The same Compose file and pipeline would apply to a college server.

1.5 Mapping of Lab Experiments to the Project

Exp	Lab topic	Application in the Hostel Management System	Section
1	Version control with Git	Shared GitHub repository, feature branches, pull requests #1 and #2, three contributors	5.1
2	Containerisation with Docker	Dockerfiles for the FastAPI backend (non-root user) and the Nginx frontend	5.2
3	Docker Compose	Two-service stack on the network hostel-network with health checks	5.3
4	Jenkins CI	Jenkinsfile pipeline: checkout, pytest unit tests with JUnit report	5.4
5	Automated deployment with Jenkins	Docker build verification, deploy, smoke test of /health	5.5
6	Agile project lifecycle (Jira)	Jira space “Hostel management”, 3 epics, 6 stories, HM Sprint 1	2
7	Monitoring with Nagios	Nagios Core host hostel-production-host with four service checks	5.6

Table 1.1: Lab experiments and where each is applied

2. Requirements and Agile Planning

2.1 User Roles

Role	Goal	Main screens
Warden (administrator)	Allocate students to rooms without errors and know the current occupancy	Dashboard, allocation form, room inventory, student registry
Student	Be placed in a suitable room and have the placement recorded	Appears in the registry with assigned room and check-in date
Development team	Plan, build, test, deploy and monitor the system	Jira, GitHub, Jenkins, Docker Desktop, Nagios

Table 2.1: User roles

2.2 Functional Requirements

- The dashboard shows total rooms, occupied beds, available vacancies and the occupancy rate, together with a connection indicator for the backend.
- The room inventory lists room number, block, type, capacity, occupancy (for example 1/2), monthly fee and status (Available, Full or Maintenance), and can be searched by room or block and filtered by status.
- The allocation form collects full name, email address, phone, emergency contact and check-in date, and offers only rooms that have an open bed and are active.
- The backend rejects an allocation when the room is full, is under maintenance, does not exist, or when the submitted data is invalid.
- The student registry lists every allocated student with contact details, assigned room, room type and check-in date, and can be searched by name or email.
- A health endpoint reports whether the service is up and whether the database is connected.

2.3 Non-Functional Requirements

- Reliability: every container has a health check, and the pipeline ends with a smoke test of the running service.
- Portability: the whole system starts with one `docker compose up -d --build` on any machine with Docker.
- Maintainability: the pipeline, the Dockerfiles and the Compose file are stored in the repository, so they are versioned and reviewed like application code.
- Security: the backend container runs as an unprivileged user, and secrets are supplied through environment variables, with only a template file (`.env.example`) kept in Git.
- Observability: availability of the frontend, the backend and the host is checked continuously by Nagios.

2.4 Agile Setup with Jira (Experiment 6)

The project was organised with Scrum in Jira. A Jira space named **Hostel management** (key HM) was created with the Summary, Timeline, Board, Backlog, Calendar, List and Forms views, and the three team members were added as assignees. The work was divided into three epics, one for each major stream of the project, and six stories that carry story points. Each

work item has an assignee and a reporter, a priority (Medium for all items) and a status that follows the board columns To Do, In Progress and Done.

Epics were used for large areas of work, and stories for deliverables that a team member can finish and show. Story points record the relative effort: the effort of the six stories adds up to 19 points. The Git commit messages use the prefixes HMS-1 to HMS-4 for the same four streams (backend, frontend, containerisation and CI/CD, and monitoring), so a commit in GitHub can be matched with the Jira item it belongs to.

2.5 Product Backlog

Key	Type	Summary	Pts	Assignee	Status
HM-1	Epic	Core Backend API & Cloud Database Architecture	—	Soham Ajwani	To Do
HM-2	Epic	Interactive Web Frontend & User Dashboard	—	Ananya Amesur	To Do
HM-3	Epic	DevOps Containerization, CI/CD Pipeline & Monitoring	—	Janvi Chattani	To Do
HM-4	Story	Setup Supabase Database Schema, Relations & Cloud Connection	3	Janvi Chattani	Done
HM-5	Story	Implement FastAPI Core Endpoints & Room Allocation Business Logic	2	Ananya Amesur	Done
HM-6	Story	Unit Testing Suite with Pytest & Automated Test Reporting	5	Soham Ajwani	Done
HM-7	Story	Design Dark Glassmorphism UI & Responsive Layout	2	Janvi Chattani	Done
HM-8	Story	Dynamic Room Allocation Form with Client-Side Validation	4	Soham Ajwani	Done
HM-9	Story	Live KPI Metrics, Filterable Directory & Alert Notifications	3	Ananya Amesur	Done

Table 2.2: Product backlog of the Jira space "Hostel management"

The three epics stay in To Do because they are containers for the stories under them; Jira closes the sprint on the stories, which is why the sprint report in Figure 2.5 counts six completed work items.

2.6 Sprint Plan

Sprint	Goal	Stories (points)
HM Sprint 1	Deliver a working room-allocation platform (database, API, tests, interface, dashboard) that is containerised, built by Jenkins and monitored by Nagios.	HM-4 (3), HM-5 (2), HM-6 (5), HM-7 (2), HM-8 (4), HM-9 (3) = 19

Table 2.3: Sprint goal and scope

Member	Epic owned	Stories	Points
Soham Ajwani (2401004)	HM-1 Core Backend API & Cloud Database Architecture	HM-6 (5), HM-8 (4)	9
Ananya Amesur (2401005)	HM-2 Interactive Web Frontend & User Dashboard	HM-5 (2), HM-9 (3)	5
Janvi Chattani (2401018)	HM-3 DevOps Containerization, CI/CD Pipeline & Monitoring	HM-4 (3), HM-7 (2)	5
Total	3 epics	6 stories	19

Table 2.4: Distribution of work across the team (as assigned in Jira)

2.7 Definition of Done

A story is moved to Done only when all of the following hold:

1. The work was done on its own Git branch (or committed with the matching HMS-n prefix) and pushed to the shared GitHub repository.
2. The change reached `main`, and the Jenkins pipeline on `main` finished with SUCCESS, including the unit tests, the Docker build check, the deployment check and the smoke test.
3. The behaviour was verified in the running application, and the service reported UP with the database CONNECTED.
4. The card was moved to the Done column of the Jira board.

The second condition links the plan to the pipeline: a story cannot honestly be closed while the build that delivers it is failing.

2.8 The Sprint in Jira

The following figures were captured from the team's Jira space and show the sprint from its first board to the sprint report. Together they document Experiment 6 and show that the work was planned, tracked and closed in the same way for every team member.

2.8.1 Board at the start of the sprint

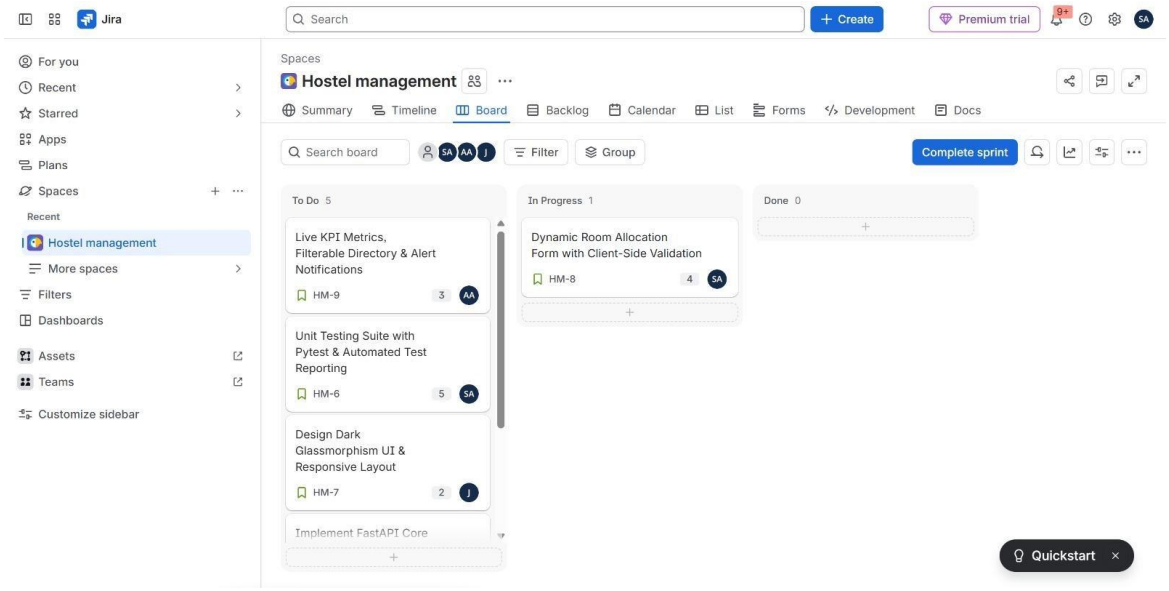


Figure 2.1: Jira board with five stories in To Do and one in progress

Figure 2.1 shows the Scrum board of the space "Hostel management" soon after the sprint started. The To Do column holds five cards (HM-9 Live KPI Metrics, HM-6 Unit Testing Suite, HM-7 Dark Glassmorphism UI, HM-5 FastAPI Core Endpoints and, below the scroll, HM-4 Supabase Schema), while the In Progress column holds only HM-8, the Dynamic Room Allocation Form. Each card shows its key, its story points (4 for HM-8) and the avatar of its assignee. The Done column is empty (Done 0). The board is the team's shared view of who is working on what, so that nobody has to ask in person which story is being worked on or what is still waiting.

2.8.2 Cards move to In Progress and Done

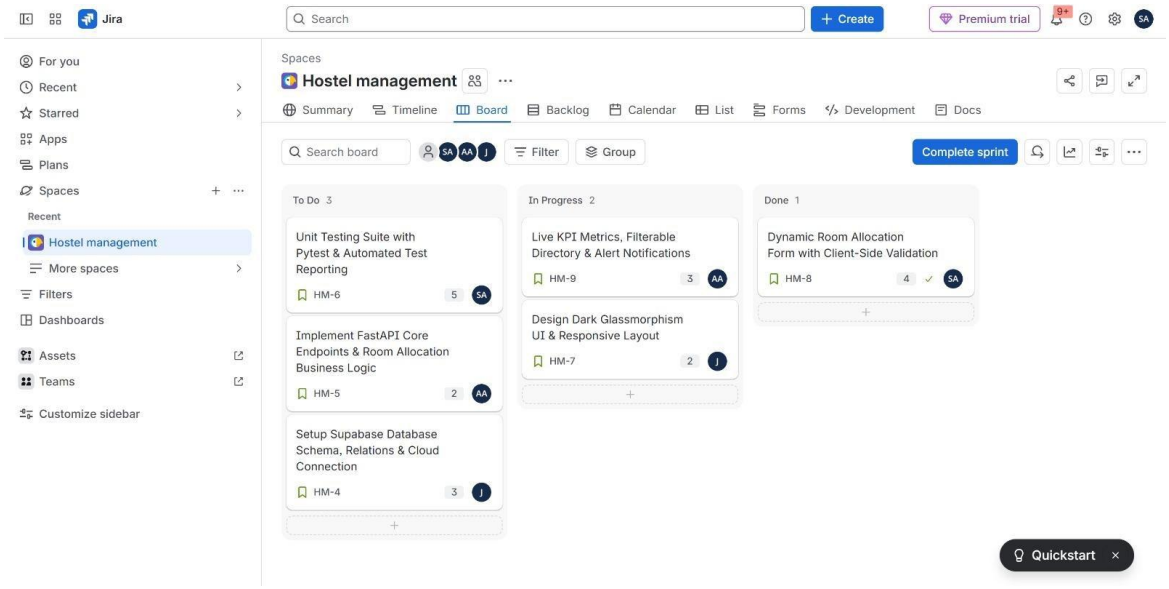


Figure 2.2: HM-8 is Done while HM-9 and HM-7 are in progress

In Figure 2.2 the first story is complete. HM-8 has moved to Done and carries a green tick, HM-9 (Live KPI Metrics) and HM-7 (Dark Glassmorphism UI) are In Progress, and To Do now holds HM-6, HM-5 and HM-4. Moving a card is how a team member reports progress: no separate status meeting is needed, because the column counts at the top of each column (To Do 3, In Progress 2, Done 1) give the sprint status at a glance.

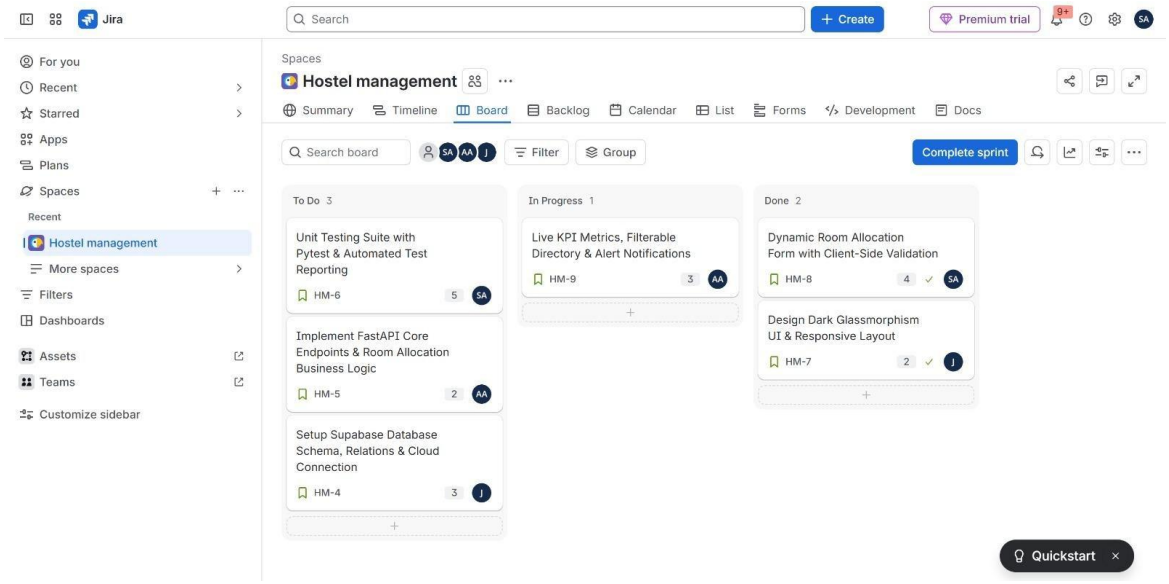


Figure 2.3: HM-7 completed; HM-9 is the only story in progress

Figure 2.3 shows the next step. HM-7 (the user-interface design) has joined HM-8 in Done, which means the dark-themed responsive layout was finished and could be used by the allocation form. Only HM-9, the live metrics story, is In Progress (To Do 3, In Progress 1, Done 2). The two completed stories are the allocation form (HM-8) and the interface design (HM-7), which are the screens the warden uses first.

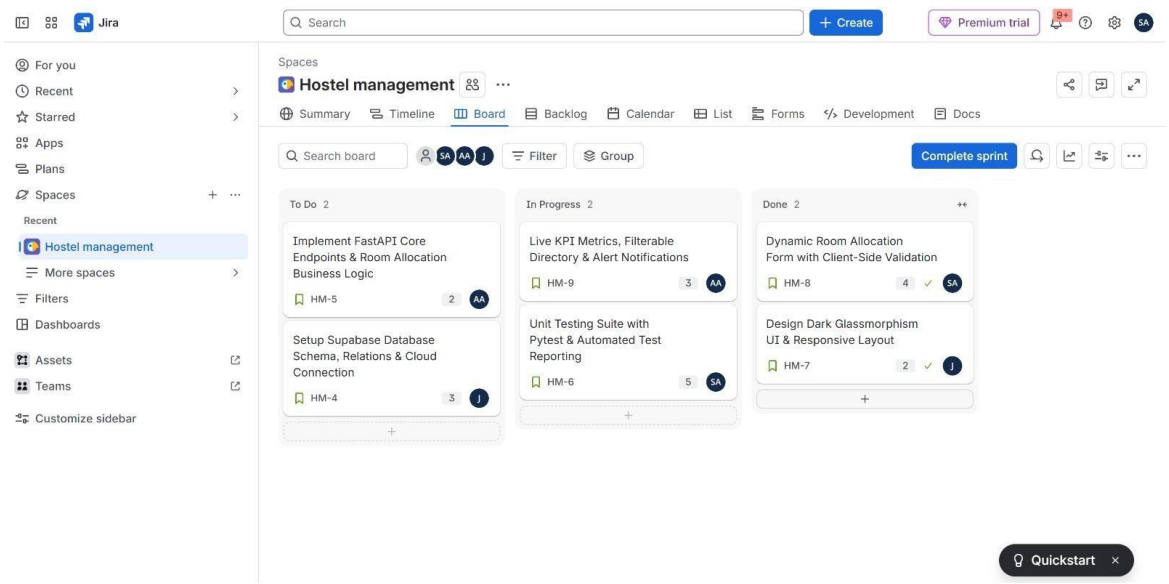


Figure 2.4: Backend and database stories remain while testing starts

In Figure 2.4 the unit-testing story HM-6 has been picked up and sits in In Progress next to HM-9, while HM-5 (FastAPI Core Endpoints) and HM-4 (Supabase Database Schema) are still in To Do. Two stories (HM-8 and HM-7) are already in Done, so the board now has two cards in each column. The remaining cards were then moved to Done one after another, as the activity feed in Figure 2.7 confirms.

2.8.3 Completing the sprint

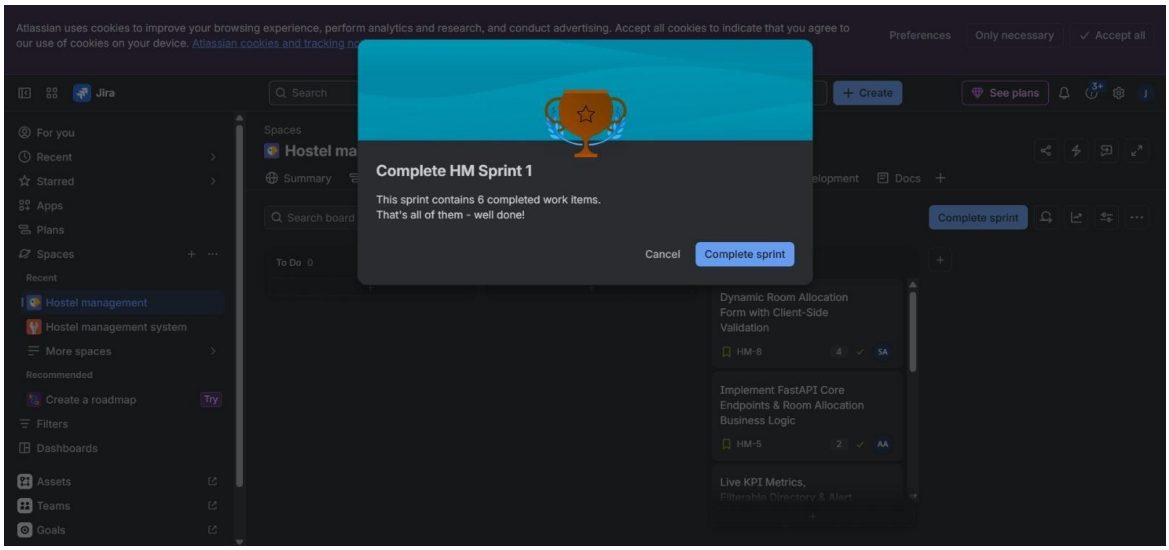


Figure 2.5: "Complete HM Sprint 1" confirmation (dark theme)

Figure 2.5 is the dialog that Jira shows when the sprint is closed with the button Complete sprint. It states that the sprint "contains 6 completed work items" and that this is all of them, so nothing has to be moved back to the backlog. This is the formal end of Sprint 1 and confirms that every one of the six stories (19 story points) was delivered. The screenshot was taken in the dark theme, which is why its colours differ from the other Jira figures.

2.8.4 Summary dashboard

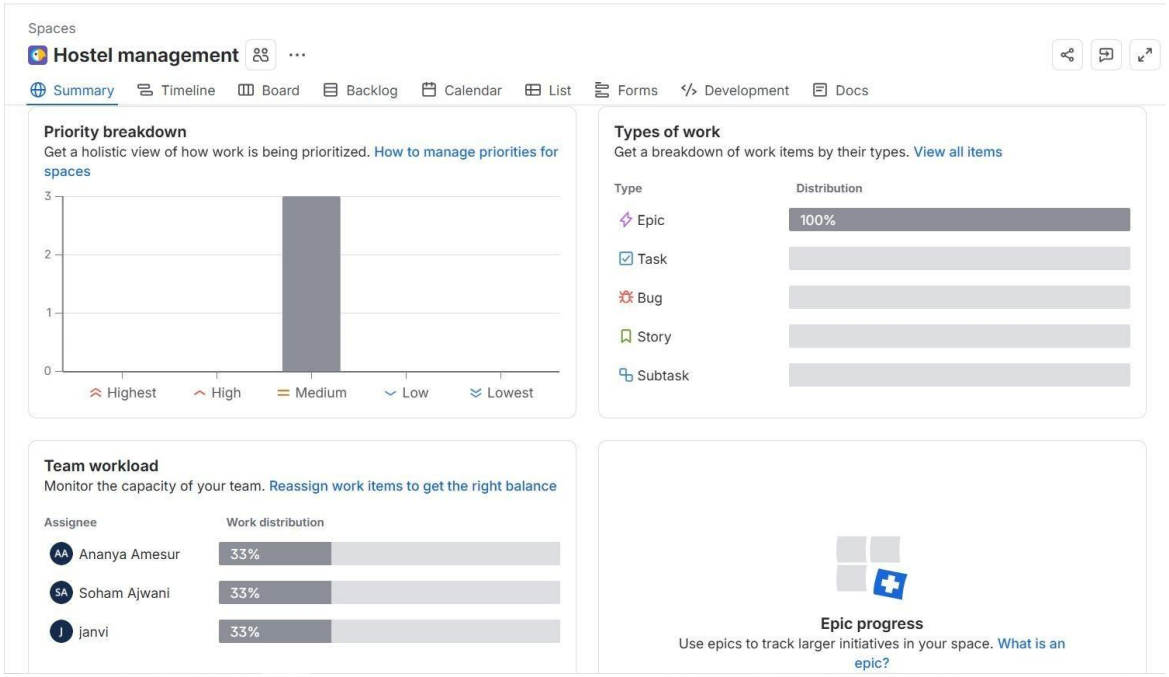


Figure 2.6: Jira summary with priority breakdown, types of work and team workload

The Summary view of Figure 2.6 gives a management view of the space. The priority chart shows that all items were given Medium priority, the types-of-work panel breaks the items down by Epic, Task, Bug, Story and Subtask, and the team-workload panel shows how many items each member is assigned. At the moment of capture the panels counted the three epics, each assigned to a different member, so Ananya Amesur, Soham Ajwani and Janvi each hold 33 % of the work. The even split shows that each team member owns one stream of the project (backend, frontend and DevOps).

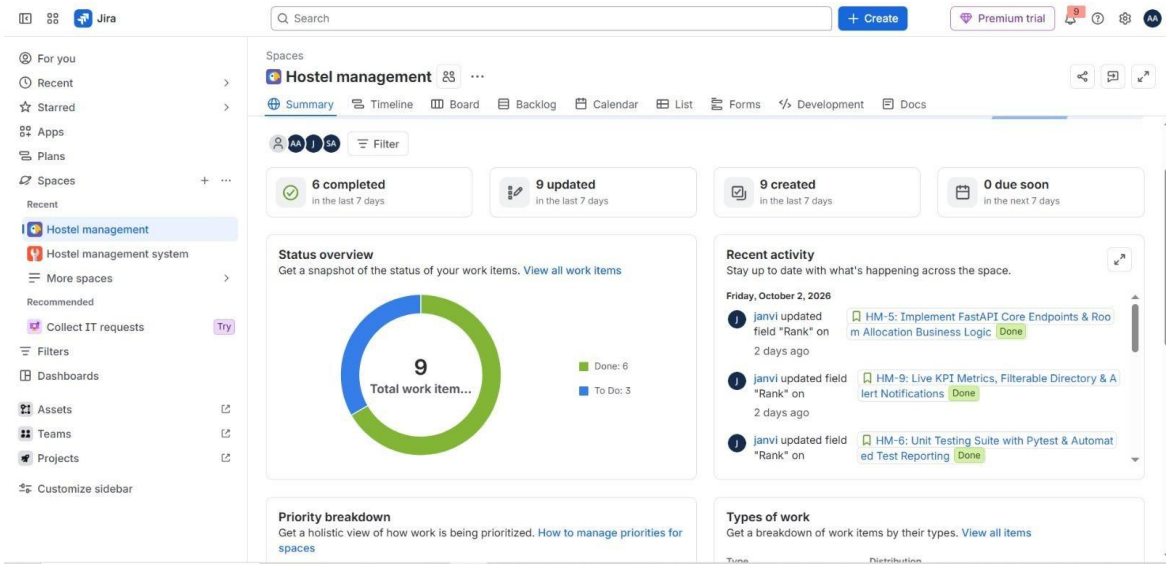


Figure 2.7: Status overview and recent activity for the whole sprint

Figure 2.7 shows the summary after the work was finished. The top row reports 6 work items completed, 9 updated and 9 created in the last seven days, with 0 due soon. The donut chart “Status overview” shows 9 work items in total, 6 in Done (green) and 3 in To Do (blue); the three still in To Do are the epics. The Recent activity feed lists the updates of Friday 2 October

2026, in which the user janvi moved HM-5 (Implement FastAPI Core Endpoints & Room Allocation Business Logic), HM-9 (Live KPI Metrics, Filterable Directory & Alert Notifications) and HM-6 (Unit Testing Suite with Pytest & Automated Test Reporting) to Done. The activity feed gives a dated, per-item audit trail of who changed what.

2.8.5 List view with assignees and status

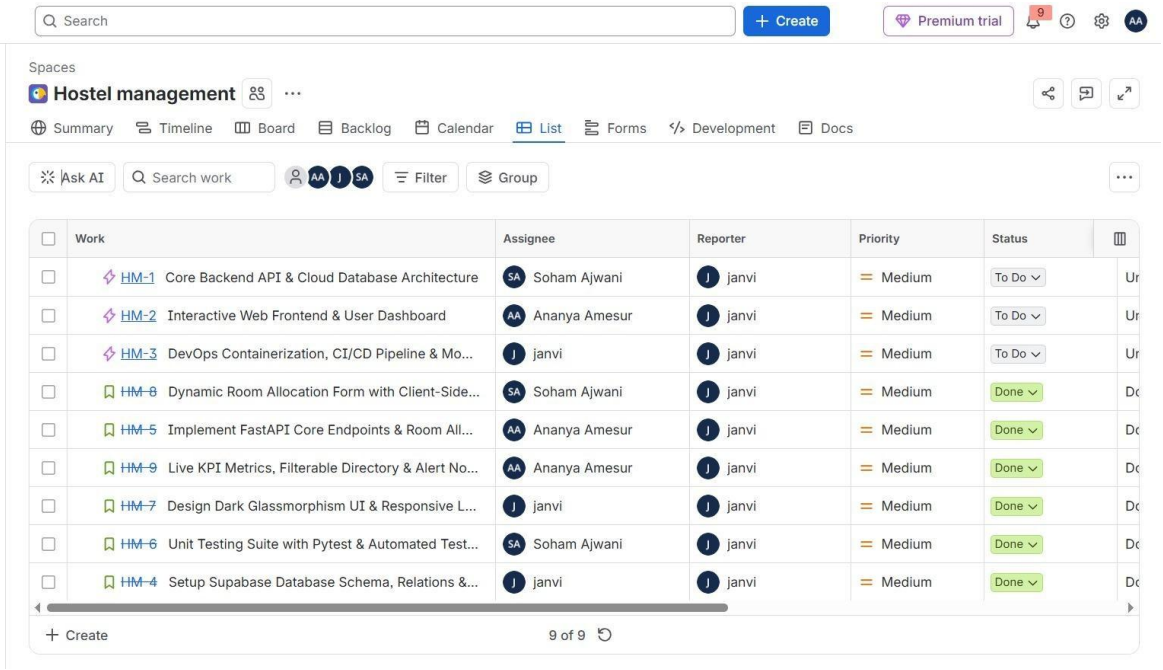


Figure 2.8: Jira list view with all nine work items

The List view in Figure 2.8 puts all nine work items ("9 of 9") in one table with columns for the work item, assignee, reporter, priority and status. HM-1, HM-2 and HM-3 are the epics, assigned to Soham Ajwani, Ananya Amesur and Janvi respectively, and all remain To Do. HM-4 to HM-9 are the stories; their keys are struck through because they are resolved, and each has the status Done. All items were created by the reporter janvi, and all have Medium priority. This table is the final evidence that the sprint goal was met: every story has an owner and is Done.

3. System Design

3.1 Architecture

The Hostel Management System is a three-part application packaged as containers (Figure 3.1). The browser loads a static web client from an Nginx container on port 80. The same Nginx instance forwards API and health requests to the FastAPI backend container on port 8000, so the browser talks to a single address. The backend keeps the room and student data in a Supabase cloud database and applies the allocation rules before anything is written.

Around the application sit the delivery and monitoring tools. The source code is kept in the GitHub repository Ananya5182/hostelmanagement. Jenkins, running on the development machine on port 8080, checks out the code from GitHub, runs the tests, verifies the Docker build, deploys the two containers with Docker Compose and smoke-tests the result. Nagios Core watches the running system from outside: it makes HTTP checks against the frontend on port 80 and the backend on port 8000, checks the room list reached through the Nginx proxy, and checks the free disk space of the host.

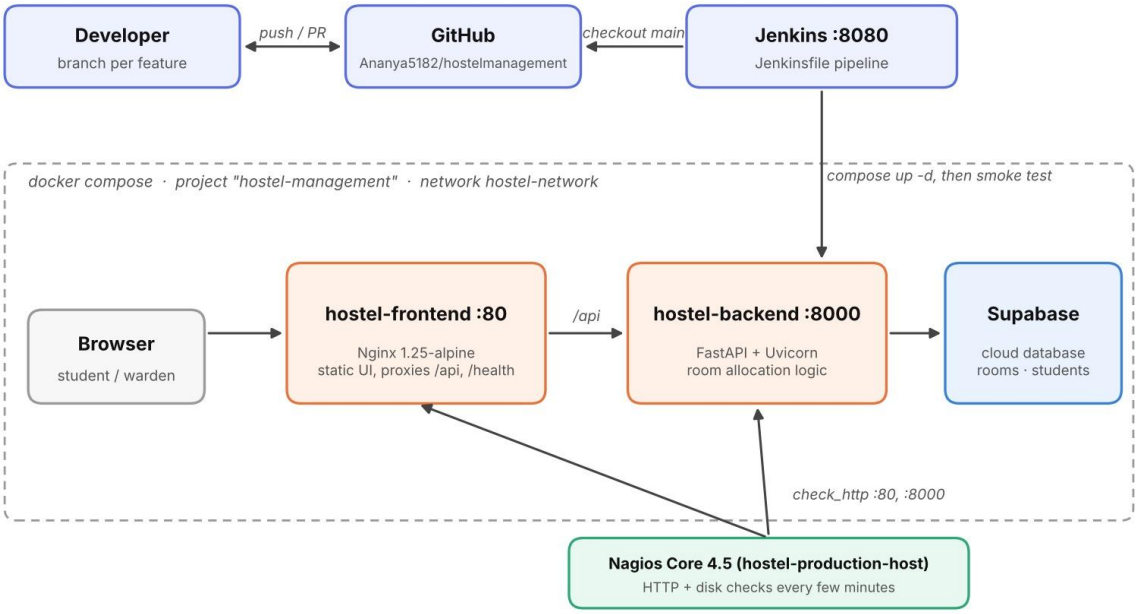


Figure 3.1: System and delivery architecture

3.2 Technology Stack

Layer	Technology	Reason for choice
Frontend	Static web client (HTML, CSS, JavaScript) with a dark glassmorphism design	No build step; client-side validation and live tables; copied directly into the Nginx image
Web server	Nginx 1.25 (alpine)	Small image (about 41 MB), serves the static files and proxies API and health requests to the backend
Backend	Python 3.11, FastAPI, Uvicorn	Typed request validation, automatic JSON handling and fast start-up
Database	Supabase (cloud database)	Managed relational database with a schema defined in database_schema.sql
Testing	pytest with JUnit XML output	API tests that need no running server; XML report that Jenkins can record

Layer	Technology	Reason for choice
Version control	Git and GitHub	Branches, pull requests and a shared history for three contributors
CI/CD	Jenkins (Jenkinsfile in the repository)	Pipeline as code: checkout, test, build, deploy, smoke test
Containers	Docker and Docker Compose	Identical environment on every machine; one command to start the stack
Monitoring	Nagios Core 4.5.12	State-based checks of every service and of the host
Project tracking	Jira	Epics, stories, sprint board, sprint report

Table 3.1: Technology stack

3.3 Database Design

Two entities carry the data (Figure 3.2): rooms and students. Each room has a number such as A-101, a block, a type (Shared Double, Single Deluxe or Shared Triple), a capacity in beds, a count of occupied beds, a monthly fee and a status, which is either active or under maintenance. A student has a name, an email address, a phone number, an emergency contact, a check-in date and the room to which he or she is assigned. One room can hold several students, up to its capacity, while each student is assigned to exactly one room. The attributes in the figure are the ones that the application displays and edits; the complete schema is kept in the file `database_schema.sql` of the repository and is applied to the Supabase database (story HM-4).

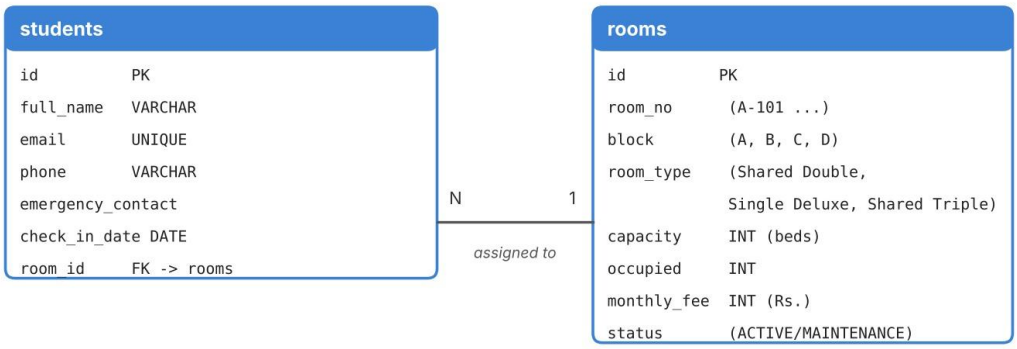


Figure 3.2: Entity-relationship diagram

3.4 Backend Operations

The backend offers a small set of operations, each of which is covered by an automated test (Section 6.1).

Operation	Purpose	Covered by test
Root endpoint	Confirms that the service is running	test_root_endpoint
Health check	Reports the status of the service and of the database connection (UP / CONNECTED, or a failure state)	test_health_check_success, test_health_check_failure
List rooms	Returns every room with block, type, capacity, occupancy, fee and status (`/api/rooms`)	test_list_rooms
List students	Returns the allocated students with their rooms	test_list_students
Allocate student	Validates the request and assigns the student to a room	test_allocate_student_success and four failure cases
Dashboard statistics	Computes total rooms, occupied beds, vacancies and occupancy rate	test_dashboard_stats

Table 3.2: Backend operations

3.5 Allocation Rules

The most important business logic is the allocation of a student to a room. The backend, not the browser, enforces the rules, so the same rules hold for any client. The commit that finalised this part is described in the repository as a transactional room allocation, which means that the student record and the occupancy change are saved together.

Situation	Result
Room exists, is active and has a free bed	Student is registered, the room occupancy increases by one, and the room becomes Full when the last bed is taken
Room is full	Request is rejected (test_allocate_student_room_full)
Room is under maintenance	Request is rejected (test_allocate_student_room_maintenance)
Room does not exist	Request is rejected as not found (test_allocate_student_room_not_found)
Required data missing or malformed	Request is rejected as invalid (test_allocate_student_invalid_payload)

Table 3.3: Allocation rules enforced by the backend

3.6 Security and Configuration Design

- The backend container runs as an unprivileged user (`appuser`, user id 1001) rather than as root.
- Settings such as database credentials are read from environment variables. The repository holds only the template `.env.example`, and `.gitignore` keeps the real `.env` file out of Git.
- Only two ports are published to the host: 80 for the web client and 8000 for the API; the containers talk to each other on the private network `hostel-network`.
- The client-side form validates input for a quick response, and the backend validates it again, because the browser cannot be trusted.

4. Application Implementation

The application consists of a static web client and a FastAPI backend. The backend contains the allocation logic and the database access; the client contains the dashboard, the allocation form with client-side validation, the room inventory table with search and filter, and the student registry. This chapter walks through the running application using the team's screenshots. The brand name shown in the interface is CampusStay, the name given to the Hostel Management & Allocation Platform.

4.1 Dashboard

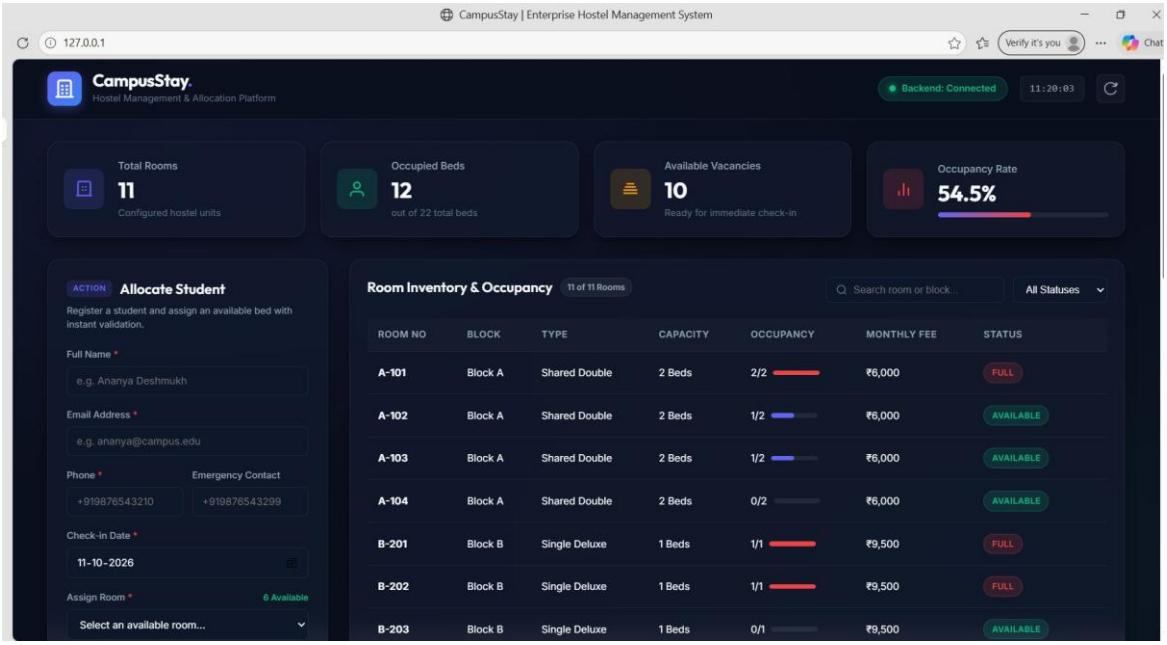


Figure 4.1: Dashboard of the Hostel Management & Allocation Platform in the browser

Figure 4.1 shows the complete dashboard opened in a browser at `localhost`. At the top right a green badge reads "Backend: Connected" and next to it a clock and a refresh button are shown. Below the title bar are four key figures, the allocation form on the left, and the room inventory table on the right. The page is dark-themed with translucent "glass" cards, the design delivered by story HM-7, and the layout adapts to the width of the window.

4.2 Live Key Figures

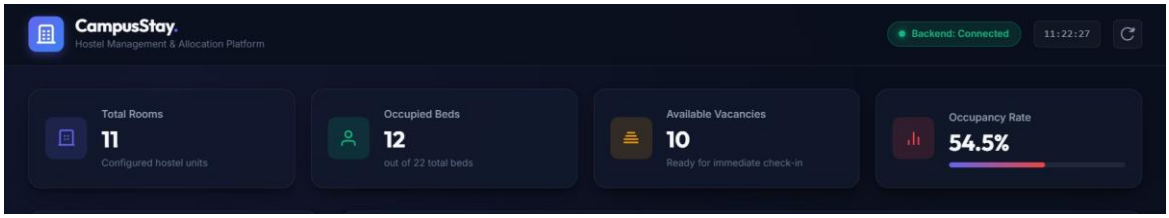


Figure 4.2: Key figures of the dashboard

The four cards in Figure 4.2 are calculated from the room data by the backend. **Total Rooms** is 11, the number of configured hostel units. **Occupied Beds** is 12 out of 22 total beds. **Available Vacancies** is 10, which is the 22 beds minus the 12 that are occupied. **Occupancy Rate** is 54.5 %, the result of 12 divided by 22, and is drawn as a progress bar. The "Backend: Connected" badge at the top right is driven by the health endpoint, so the warden knows at

once if the service or its database is unavailable. These figures are delivered by story HM-9 (Live KPI Metrics).

4.3 Allocation Form and Room Inventory

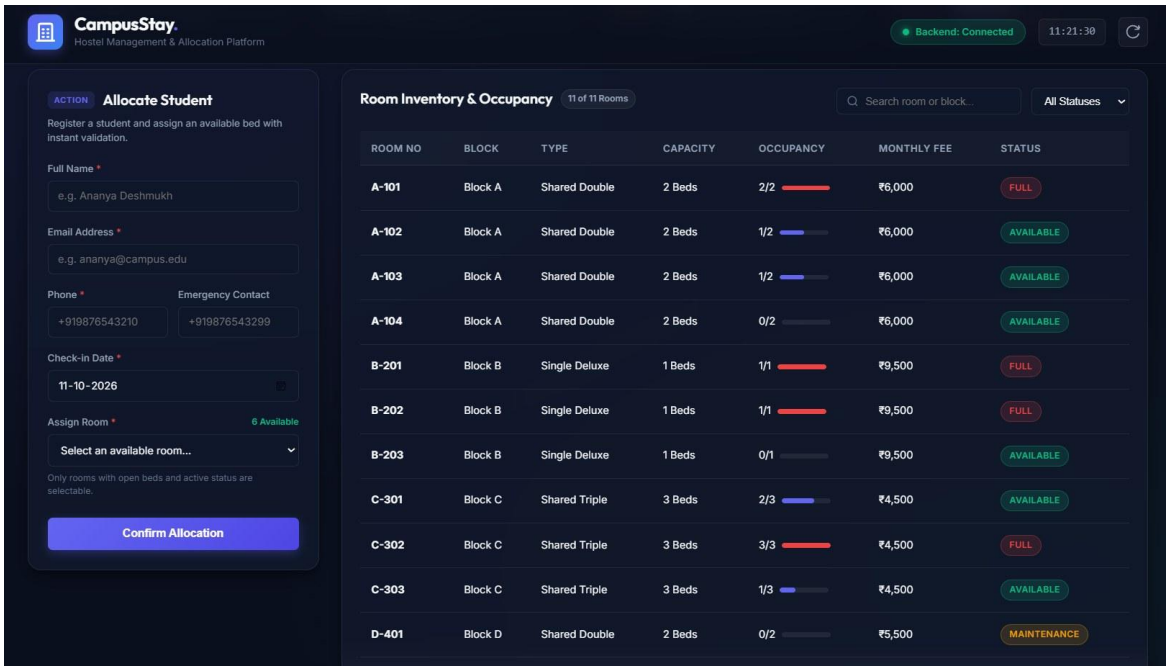


Figure 4.3: Allocate Student form and Room Inventory & Occupancy table

The left panel of Figure 4.3 is the **Allocate Student** form. It asks for the full name, email address, phone, emergency contact and check-in date (pre-filled with the current date), and has a drop-down list "Assign Room" with the note that only rooms with open beds and active status can be selected. The label on the drop-down shows that six rooms are currently available. The form is validated in the browser (story HM-8) before it is sent, and the button Confirm Allocation submits it to the backend, where the rules of Table 3.3 are applied.

The right panel is the **Room Inventory & Occupancy** table, which lists all 11 rooms of the four blocks. For every room it shows the room number, block, type, capacity, occupancy as a fraction with a coloured bar, the monthly fee and a status badge. Rooms A-101, B-201, B-202 and C-302 are Full (red); A-102, A-103, A-104, B-203, C-301 and C-303 are Available (green); and D-401 is under Maintenance (amber) and therefore cannot be chosen in the form. The fee depends on the room type: ₹6,000 for a shared double in Block A, ₹9,500 for a single deluxe in Block B, ₹4,500 for a shared triple in Block C and ₹5,500 for a shared double in Block D. A search box (room or block) and a status filter above the table narrow the list.

4.4 Allocated Students Registry

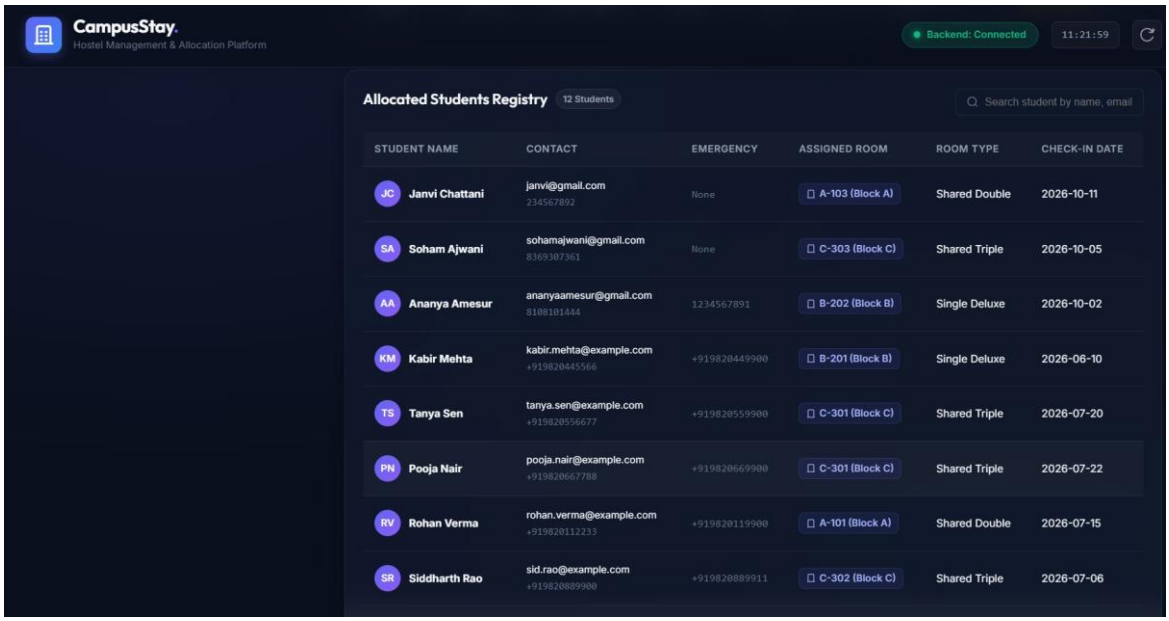


Figure 4.4: Allocated Students Registry

Figure 4.4 shows the **Allocated Students Registry**, which lists 12 students, the same number as the occupied beds on the dashboard. Each row gives the student's name with initials, the email address and phone number, the emergency contact, the assigned room with its block, the room type and the check-in date. The first three rows are the three members of the project team: Janvi Chattani in A-103 (check-in 2026-10-11), Soham Ajwani in C-303 (2026-10-05) and Ananya Amesur in B-202 (2026-10-02). The remaining rows are sample students with example.com addresses, such as Kabir Mehta in B-201 and Tanya Sen in C-301. A search box filters the registry by name or email. Because the registry and the occupancy figures come from the same data, the count of 12 students and the 12 occupied beds can be cross-checked, which is one of the verifications in Section 6.3.

4.5 Backend Logic

The FastAPI application is started by Uvicorn inside the backend container. It exposes the operations of Table 3.2, validates incoming data, reads and writes the Supabase database, and answers the health probe with a small JSON document (`{"status": "UP", "database": "CONNECTED"}`) that the Docker health check, the Jenkins smoke test and the Nagios check all rely on. Because the same endpoint serves three different tools, a failure of the database shows up in all of them at once. The tests of Chapter 6 call the application directly, so the business rules can be tested without starting a browser or a server.

5. DevOps Implementation

This chapter describes how each lab experiment was applied to the Hostel Management System. Figure 5.1 summarises the delivery pipeline that ties them together: work starts as a Jira story, is committed to a branch in Git, reaches `main` through a pull request, is tested by Jenkins, built and deployed as Docker containers, and is then watched by Nagios. Experiment 6 (Jira) was described in Chapter 2; Sections 5.1 to 5.6 cover Experiments 1 to 5 and 7, and Section 5.7 explains how CI/CD is achieved from end to end.

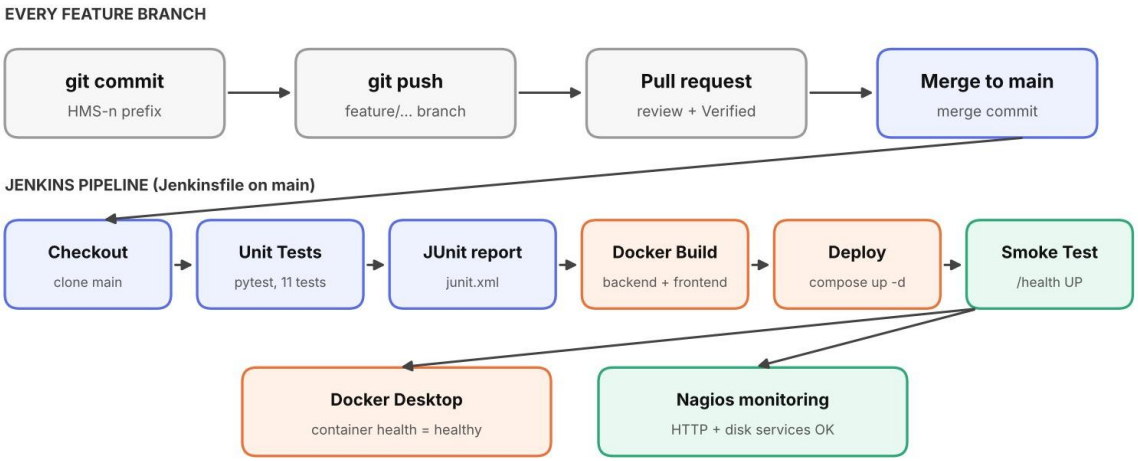


Figure 5.1: Delivery pipeline from commit to monitoring

5.1 Version Control with Git and GitHub (Experiment 1)

All source code, the Dockerfiles, the Compose files, the Jenkinsfile, the Nagios configuration and the helper scripts are kept in one GitHub repository, `Ananya5182/hostelmanagement`. The team follows one rule: each stream of work is developed on its own branch, and a branch reaches `main` only through a pull request. Every commit message begins with a prefix that names the stream, HMS-1 for the backend, HMS-2 for the frontend, HMS-3 for Docker and CI/CD and HMS-4 for monitoring, followed by a conventional type such as `feat` or `docs`.

5.1.1 Repository set-up and first push

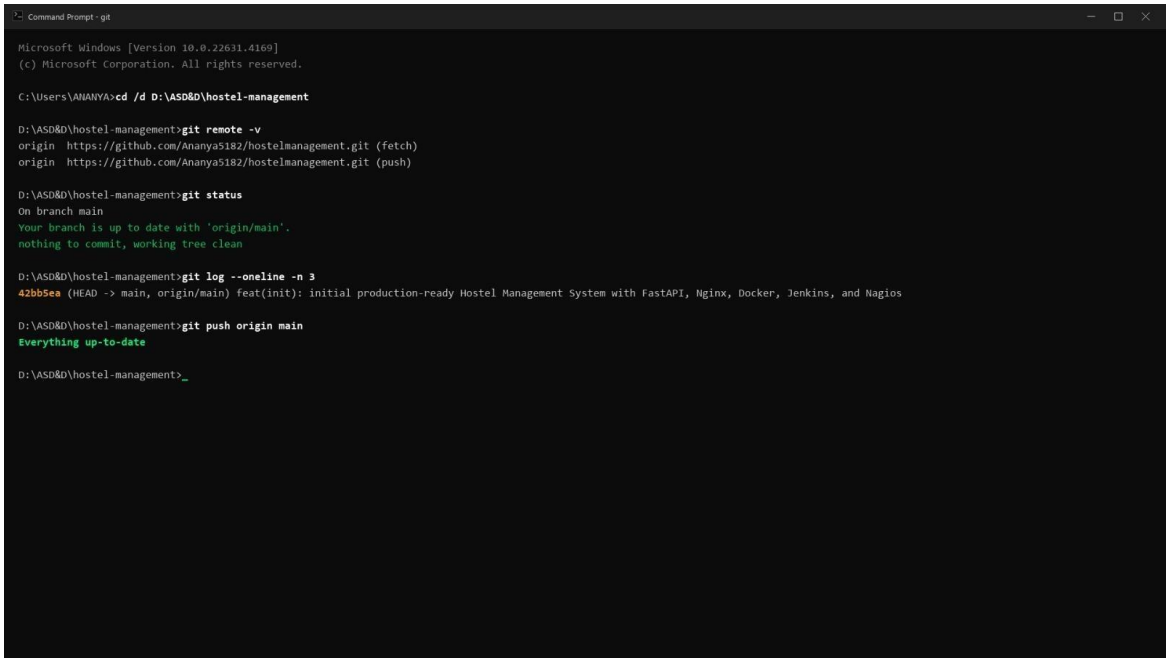


Figure 5.2: Remote configuration, status, log and push in the project folder

Figure 5.2 shows the basic Git workflow in a Command Prompt, in the project folder `D:\ASD&D\hostel-management`. The command `git remote -v` shows that the remote named origin points to `https://github.com/Ananya5182/hostelmanagement.git` for both fetch and push, which links the local folder to the shared repository. `git status` reports "On branch main", "Your branch is up to date with origin/main" and "nothing to commit, working tree clean", so the local copy equals the shared copy. `git log --oneline -n 3` lists the initial commit `42bb5ea`, with the message "feat(init): initial production-ready Hostel Management System with FastAPI, Nginx, Docker, Jenkins, and Nagios", and `git push origin main` answers "Everything up-to-date". This is the starting point for the pipeline: Jenkins later checks out exactly this branch from this remote.

5.1.2 Branches and merge history

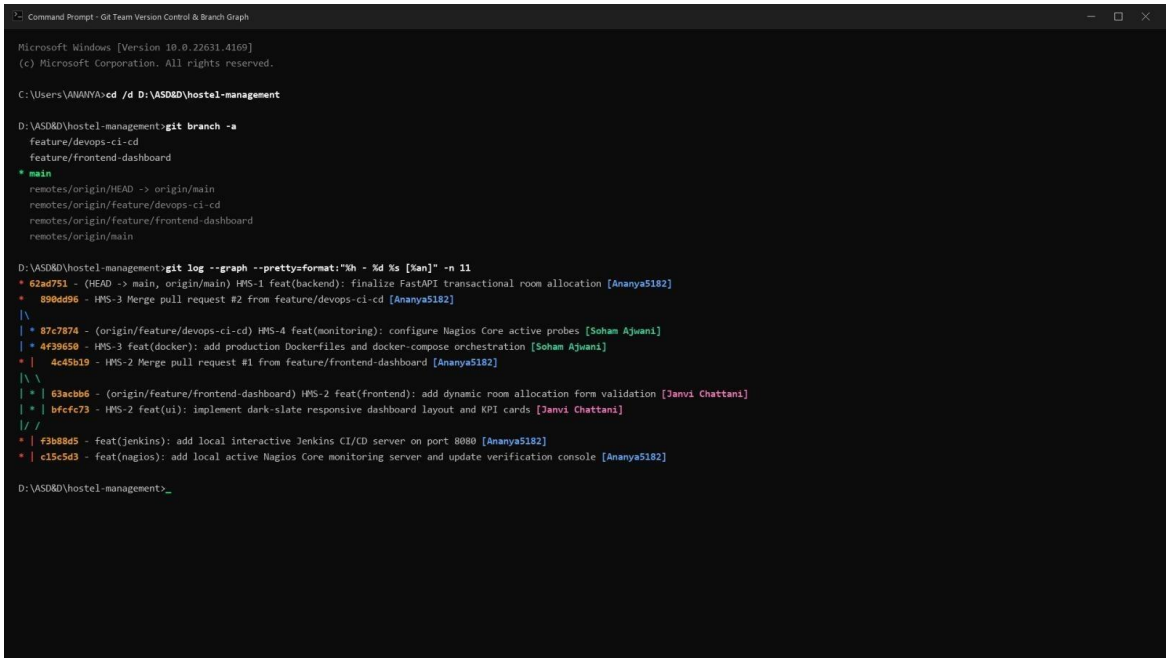


Figure 5.3: Local and remote branches with the commit graph of the three authors

In Figure 5.3 `git branch -a` lists the local branches `main`, `feature/devops-ci-cd` and `feature/frontend-dashboard` together with their remote-tracking branches under `remotes/origin`. The command `git log --graph --pretty=format:"%h - %d %s [%an]" -n 11` then draws the history with the author name of every commit in brackets. The graph shows two side branches that split off `main` and come back through merge commits:

- **feature/frontend-dashboard**: two commits by Janvi Chattani (`bfcfc73` HMS-2 dark-slate responsive dashboard layout and KPI cards, and `63acbb6` HMS-2 dynamic room allocation form validation), merged by "HMS-2 Merge pull request #1" (`4c45b19`).
- **feature/devops-ci-cd**: two commits by Soham Ajwani (`4f39650` HMS-3 production Dockerfiles and docker-compose orchestration, and `87c7874` HMS-4 Nagios Core active probes), merged by "HMS-3 Merge pull request #2" (`890dd96`).

On `main` itself Ananya Amesur's commits (shown under the repository account Ananya5182) add the Nagios server (`c15c5d3`), the interactive Jenkins server on port 8080 (`f3b88d5`) and finally "HMS-1 feat(backend): finalize FastAPI transactional room allocation" (`62ad751`). The graph therefore shows parallel work by three people that was joined without losing anyone's history, which is the purpose of branching.

Commit	Stream	Message	Author
bfcfc73	HMS-2	feat(ui): implement dark-slate responsive dashboard layout and KPI cards	Janvi Chattani
63acbb6	HMS-2	feat(frontend): add dynamic room allocation form validation and reactive tables	Janvi Chattani
4f39650	HMS-3	feat(docker): add production Dockerfiles and docker-compose orchestration	Soham Ajwani
87c7874	HMS-4	feat(monitoring): configure Nagios Core active probes for disk, frontend, and backend health	Soham Ajwani
c15c5d3	—	feat(nagios): add local active Nagios Core monitoring server and update verification console	Ananya Amesur
f3b88d5	—	feat(jenkins): add local interactive Jenkins CI/CD server on port 8080	Ananya Amesur

Commit	Stream	Message	Author
62ad751	HMS-1	feat(backend): finalize FastAPI transactional room allocation and database integration	Ananya Amesur
42bb5ea	—	feat(init): initial production-ready Hostel Management System with FastAPI, Nginx, Docker, Jenkins, and Nagios	Ananya Amesur

Table 5.1: Commits of the three authors

5.1.3 The repository on GitHub

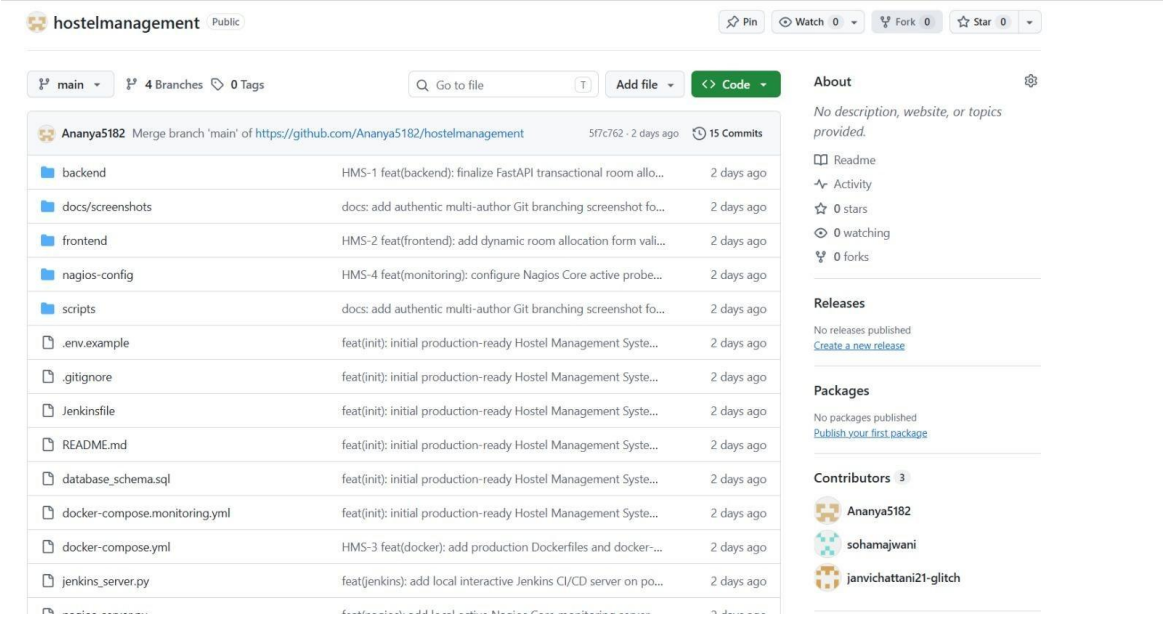


Figure 5.4: GitHub repository page of hostelmanagement

Figure 5.4 shows the repository `hostelmanagement` on GitHub. The header reports 4 branches, 0 tags and 15 commits on `main`, and the right-hand panel lists three contributors: Ananya5182, sohamajwani and janvichattani21-glitch, one for each team member. The file list shows how the project is organised: `backend`, `frontend`, `nagios-config`, `scripts` and `docs/screenshots` folders and, at the top level, `.env.example`, `.gitignore`, `Jenkinsfile`, `README.md`, `database_schema.sql`, `docker-compose.yml`, `docker-compose.monitoring.yml` and `jenkins_server.py`. The Jenkinsfile and the Compose files sit next to the application code, so the way the software is built and deployed is versioned together with the software itself. This is the basis for the CI/CD of Section 5.7.

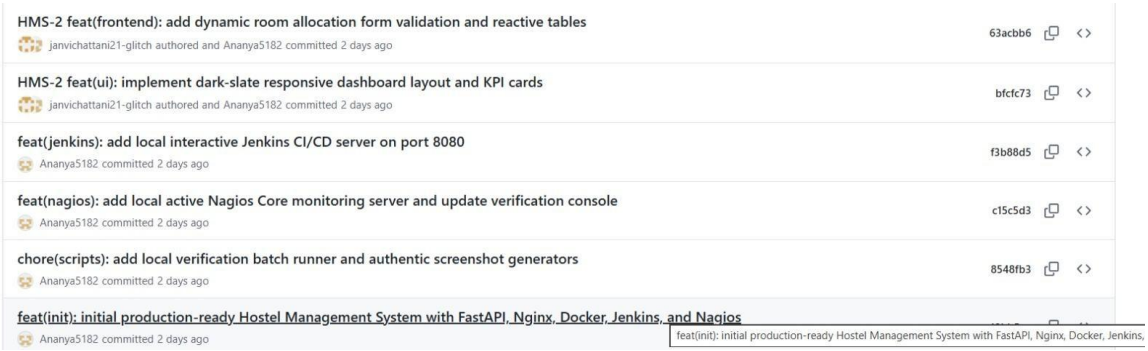


Figure 5.5: Commit list showing authors and committer

Figure 5.5 lists recent commits with the identity of the author and the committer. Janvi's commits read "janvichattani21-glitch authored and Ananya5182 committed", because the work

of the three members was pushed to GitHub through the repository account, while Git still records the original author of every change. This keeps every member's contribution visible even though the push came from one machine.

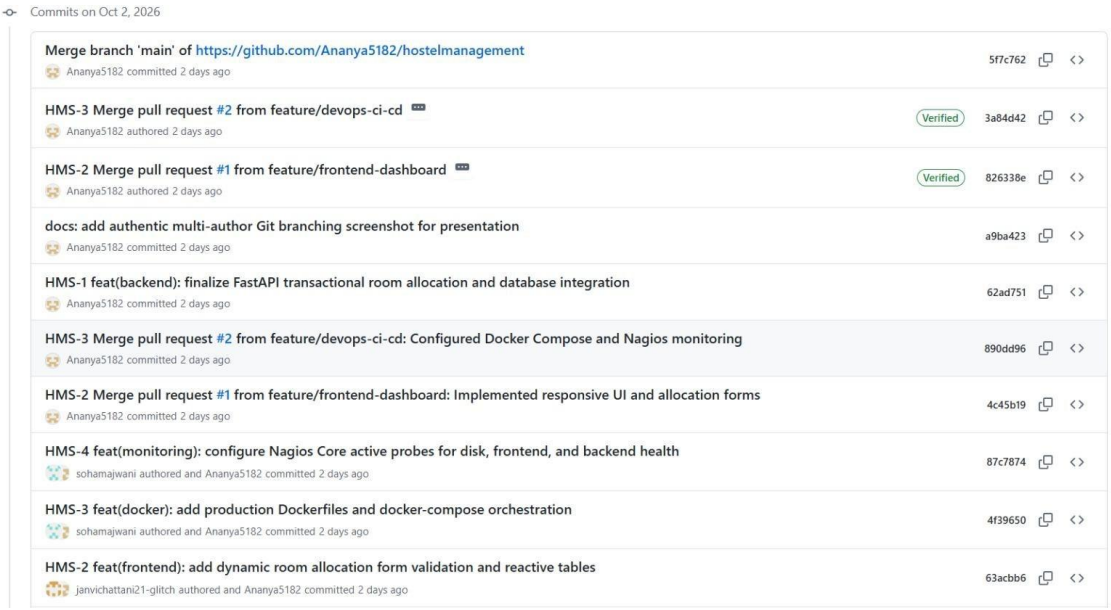


Figure 5.6: Commits of 2 October 2026 including the two merged pull requests

Figure 5.6 lists the commits of 2 October 2026 in order. It includes "HMS-2 Merge pull request #1 from feature/frontend-dashboard" and "HMS-3 Merge pull request #2 from feature/devops-ci-cd", both with a green **Verified** badge, which means GitHub has checked the signature of the merge. The list also contains the feature commits of Janvi and Soham, the backend commit HMS-1, a documentation commit that adds the Git branching screenshot, and a final "Merge branch 'main'" that joins the local and the remote history. Each merge produces a single commit that records which branch was joined, so the history shows when and why each stream entered `main`. A merged pull request is the trigger event for the pipeline: once the code is on `main`, Jenkins can build the exact version that was approved.

5.2 Containerisation with Docker (Experiment 2)

Each tier of the application has its own Dockerfile, and each is built into an image of its own. The build output of both images is shown in Figure 5.7.

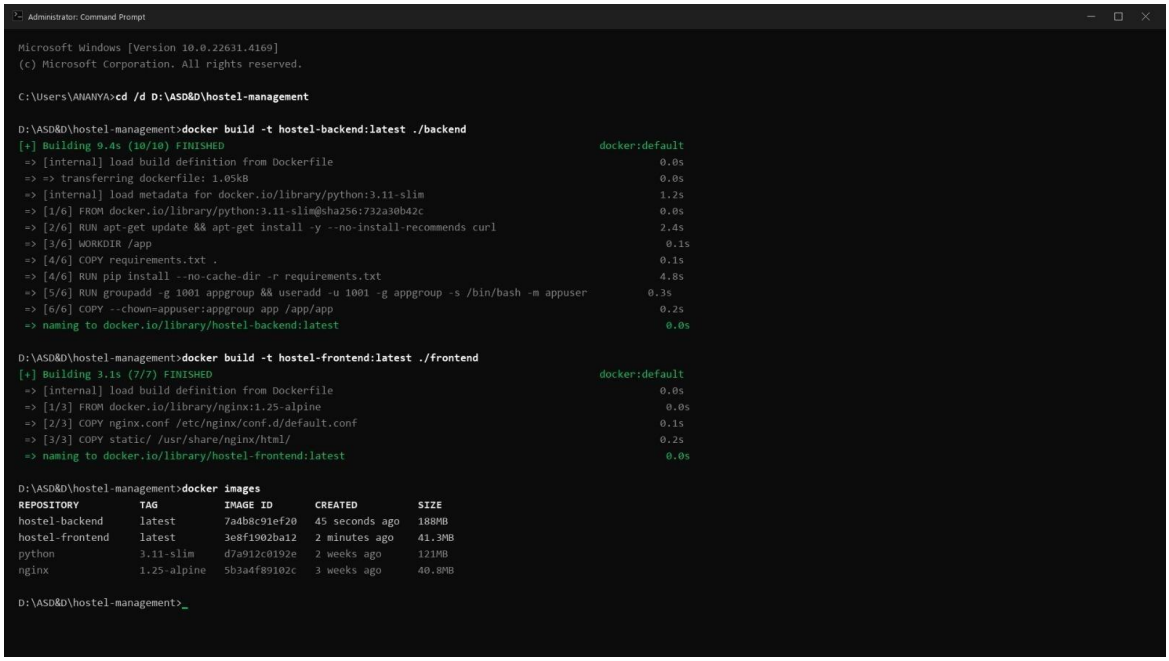


Figure 5.7: Docker build of the backend and frontend images and the resulting image list

The first command, `docker build -t hostel-backend:latest ./backend`, finished in 9.4 seconds in 10 steps. The steps that Docker prints correspond to the following instructions of the backend Dockerfile:

```
FROM python:3.11-slim
RUN apt-get update && apt-get install -y --no-install-recommends curl
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
RUN groupadd -g 1001 appgroup && useradd -u 1001 -g appgroup -s /bin/bash -m appuser
COPY --chown=appuser:appgroup app /app/app
```

The base image `python:3.11-slim` keeps the image small. The list of dependencies is copied and installed before the application code, so the slow `pip install` layer is reused from the cache whenever only the source code changes. A non-privileged user `appuser` (id 1001) is created and owns the application files, so the service does not run as root, and `curl` is installed so that the container can probe its own health endpoint. The second command, `docker build -t hostel-frontend:latest ./frontend`, took 1.1 seconds in 7 steps and uses an even smaller recipe:

```
FROM nginx:1.25-alpine
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY static/ /usr/share/nginx/html/
```

The frontend image starts from `nginx:1.25-alpine`, replaces the default server configuration with the project's `nginx.conf` (which serves the pages and forwards the API and health requests to the backend) and copies the static pages into the folder that Nginx serves. The final command, `docker images`, lists the result:

Repository	Tag	Size	Role
hostel-backend	latest	188 MB	FastAPI service on Python 3.11
hostel-frontend	latest	41.3 MB	Nginx with the static web client
python	3.11-slim	121 MB	Base image of the backend

Repository	Tag	Size	Role
nginx	1.25-alpine	40.8 MB	Base image of the frontend

Table 5.2: Docker images built for the project

The table shows how little each Dockerfile adds. The frontend image is only about half a megabyte larger than the Nginx base image, because it contains nothing but static files; the backend image adds roughly 67 MB of libraries and application code to the Python base. Because the images are built from files in the repository, any team member, and Jenkins, can recreate exactly the same environment with the same command, which removes the "it works on my machine" problem.

5.3 Multi-Container Orchestration with Docker Compose (Experiment 3)

The backend and frontend must run together, with the right ports, the right network and the right start-up order, so they are described once in `docker-compose.yml` and started with a single command. Figure 5.8 shows that command and its verification.

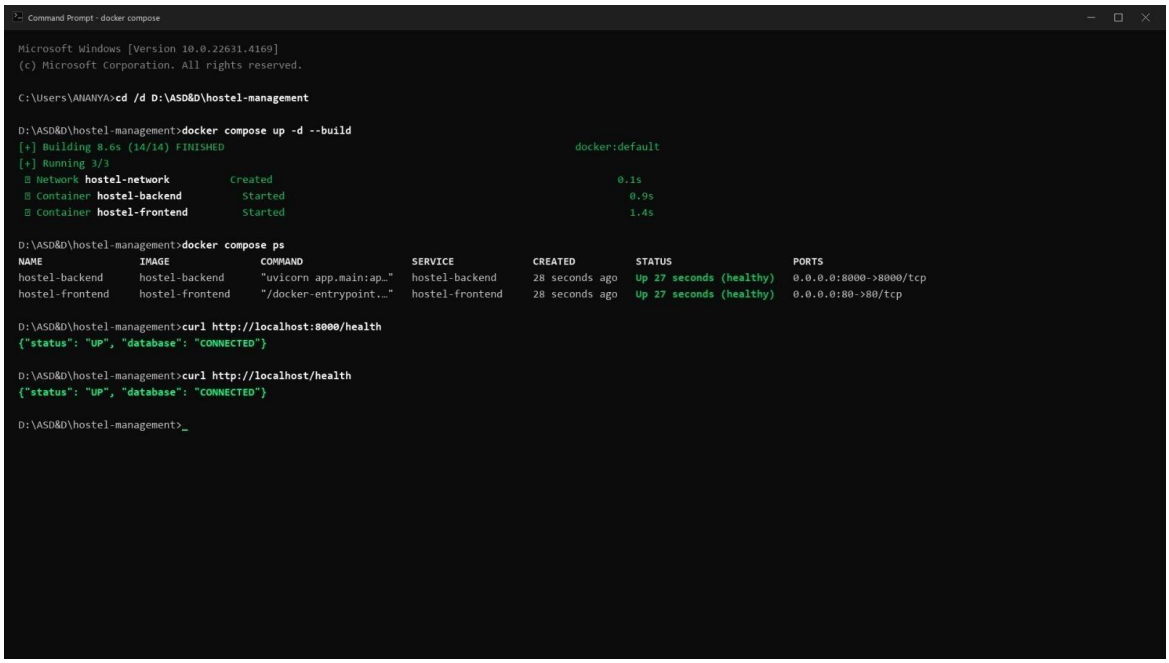


Figure 5.8: docker compose up, docker compose ps and the health requests

The command `docker compose up -d --build` rebuilt the images (14 steps in 8.6 seconds), created a network named **hostel-network** and started the containers `hostel-backend` and `hostel-frontend` in the background ("Running 3/3": one network and two containers). `docker compose ps` then shows both containers with the status "Up 27 seconds (healthy)": the backend runs `uvicorn app.main:app` and publishes port 8000, and the frontend runs the Nginx entrypoint and publishes port 80. The word "healthy" means that the health check defined for each container is passing, not just that the process has started.

Two `curl` commands then prove that the whole chain works. `curl http://localhost:8000/health` talks to the backend directly and returns `{"status": "UP", "database": "CONNECTED"}`, which shows that the service is up and can reach the Supabase database. `curl http://localhost/health` goes through the Nginx container on port 80 and returns the same answer, which proves that the reverse proxy forwards requests to the backend over the private network. The two services and their roles are summarised below.

Service	Image	Host port	Notes
hostel-backend	hostel-backend:latest	8000	FastAPI with Uvicorn, health check, connects to Supabase
hostel-frontend	hostel-frontend:latest	80	Nginx serving the web client and proxying API and /health to the backend
hostel-network	(bridge network)	—	Private network that lets the frontend reach the backend by name

Table 5.3: Services in the Compose stack

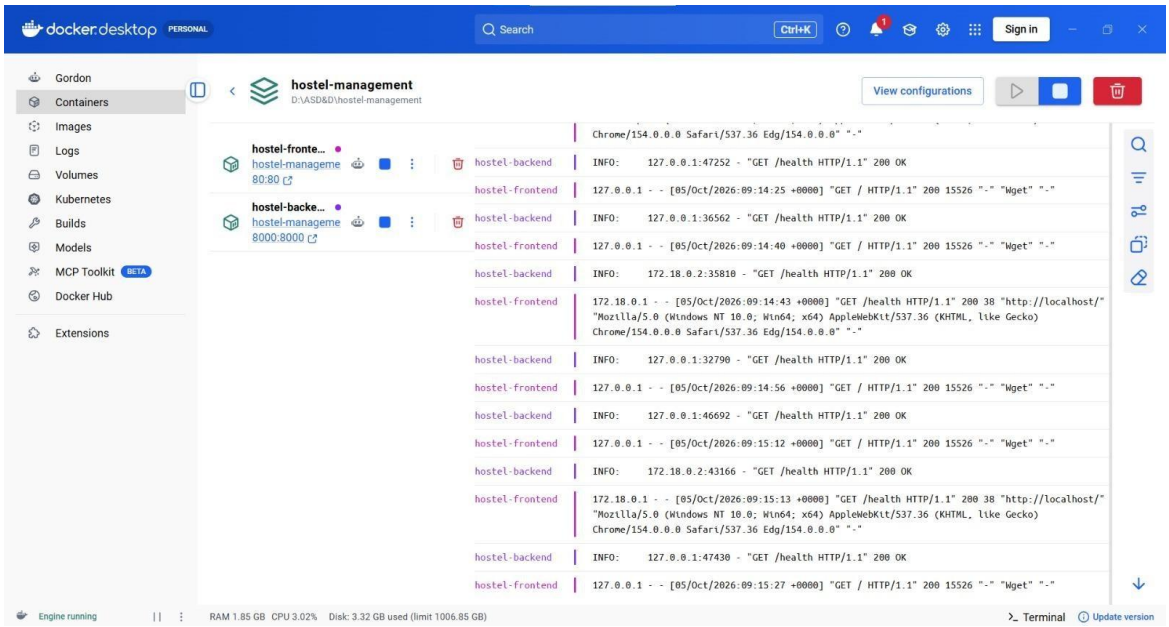


Figure 5.9: Docker Desktop showing the hostel-management stack and live container logs

Figure 5.9 shows the same stack in Docker Desktop. The group `hostel-management` (located at `D:\ASD&D\hostel-management`) contains two containers: `hostel-frontend` with the port mapping 80:80 and `hostel-backend` with 8000:8000, and both are running. The log pane merges the output of the two services. The lines `hostel-backend | INFO 127.0.0.1 … "GET /health HTTP/1.1" 200 OK` repeat about every fifteen seconds: this is the health check that Docker itself runs inside the backend container. The `hostel-frontend` lines record Nginx requests such as `GET / HTTP/1.1` and `GET /health HTTP/1.1` with status 200, some from the container's own check (user agent Wget) and some from a browser (Chrome/Edge) that was opened on the dashboard. The footer of the window reports that the engine is running with 1.85 GB of RAM and about 3 % CPU in use. This screenshot is evidence that the containers keep running and keep passing their health checks after deployment, which is what the monitoring of Section 5.6 relies on.

5.4 Continuous Integration with Jenkins (Experiment 4)

Jenkins automates what the team would otherwise do by hand after every change: fetch the code, run the tests, check that the images can be built and confirm that the service starts. The pipeline is declared in the file `Jenkinsfile` in the root of the repository, so it is versioned and reviewed like any other code. A local Jenkins server (`jenkins_server.py`, added in commit `f3b88d5`) provides the Jenkins console on port 8080 of the development machine. Figure 5.10 is the console output of one complete run.

```
Started by user Ananya (Manual Trigger)
Running in Durability level: MAX_SURVIVABILITY
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins local worker: D:\ASD&D\hostel-management
[Pipeline] stage (Checkout)
[+] Checking out repository: https://github.com/Ananya5182/hostelmanagement.git (Branch: main)
[Pipeline] stage (Unit Tests)
============================= test session starts =============================
platform win32 -- Python 3.13.3, pytest-9.1.1, pluggy-1.6.0 -- C:\Users\ANANYA\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: D:\ASD&D\hostel-management
plugins: anyio-4.14.2, langsmith-0.12.1, mock-3.16.0
collecting ... collected 11 items

backend/tests/test_main.py::test_root_endpoint PASSED                    [  9%]
backend/tests/test_main.py::test_health_check_success PASSED             [ 18%]
backend/tests/test_main.py::test_health_check_failure PASSED             [ 27%]
backend/tests/test_main.py::test_list_rooms PASSED                       [ 36%]
backend/tests/test_main.py::test_list_students PASSED                    [ 45%]
backend/tests/test_main.py::test_allocate_student_success PASSED         [ 54%]
backend/tests/test_main.py::test_allocate_student_room_full PASSED       [ 63%]
backend/tests/test_main.py::test_allocate_student_room_maintenance PASSED [ 72%]
backend/tests/test_main.py::test_allocate_student_room_not_found PASSED  [ 81%]
backend/tests/test_main.py::test_allocate_student_invalid_payload PASSED [ 90%]
backend/tests/test_main.py::test_dashboard_stats PASSED                  [100%]

---- generated xml file: D:\ASD&D\hostel-management\test-reports\junit.xml ----
============================== 11 passed in 1.62s ==============================

Recording test results in JUnit format: test-reports/junit.xml
[Pipeline] stage (Docker Build)
[+] Verified Dockerfile syntax and container configurations.
[Pipeline] stage (Deploy)
[+] Verified services on http://localhost:8000
[Pipeline] stage (Smoke Test)
[+] Health probe status: UP (Database: CONNECTED)
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Figure 5.10: Jenkins console output of a successful pipeline run

The console output can be read from top to bottom as the story of one build:

- **Started by user Ananya (Manual Trigger).** The run was started by hand from the Jenkins interface. The line "Running in Durability level: MAX_SURVIVABILITY" shows that the pipeline is configured to keep its state if the server restarts.
- **[Pipeline] node: Running on Jenkins local worker: D:\ASD&D\hostel-management.** The build runs on a worker on the development machine, inside the project folder.
- **Stage (Checkout).** The line "Checking out repository: https://github.com/Ananya5182/hostelmanagement.git (Branch: main)" shows the integration with Git: the pipeline always builds what is on `main` in GitHub, not what happens to be on someone's disk.
- **Stage (Unit Tests).** pytest 9.1.1 starts on Python 3.13.3 under Windows, with the project root `D:\ASD&D\hostel-management` and the cache folder `.pytest_cache`. It collects 11 items and runs them one by one; each prints PASSED with the running percentage (9 % to 100 %). The run ends with "11 passed in 1.62s".
- **JUnit report.** The line "generated xml file: …\test-reports\junit.xml" followed by "Recording test results in JUnit format" means the test results are stored as a standard XML report, which Jenkins can show as a trend and use to decide whether a build passes.

The eleven tests are listed in Section 6.1. Among them are the rules of Table 3.3: a successful allocation is accepted, while a full room, a room under maintenance, an unknown room and an invalid request are all rejected. The tests therefore protect the most important behaviour of the system each time the pipeline runs. If any test failed, the stage would fail and, as in any declarative Jenkins pipeline, the later stages (build, deploy, smoke test) would not run, so a broken change is stopped before it is released.

Stage	What it does (as shown in the console)	Purpose
Checkout	Clones the repository https://github.com/Ananya5182/hostelmanagement.git, branch main	Build exactly the code that is on main
Unit Tests	Runs pytest on backend/tests/test_main.py: 11 items, 11 passed in 1.62 s; writes test-reports/junit.xml	Catch defects before anything is built or deployed
Docker Build	Verifies the Dockerfile syntax and the container configurations	Make sure the images can be built from the repository
Deploy	Verifies that the services are available at http://localhost:8000	Confirm that the stack is serving
Smoke Test	Calls the health probe: status UP (Database: CONNECTED)	Prove that the deployed build works end to end

Table 5.4: Stages of the Jenkins pipeline

5.5 Automated Deployment with Jenkins (Experiment 5)

Continuous integration ends when the code has been tested; continuous deployment goes on to put the tested code into service and to check that it is working. In the project this is the second half of the same pipeline run (Figure 5.10), and it automates the manual steps that were carried out in Experiment 3:

```
[Pipeline] stage (Docker Build)
[+] Verified Dockerfile syntax and container configurations.
[Pipeline] stage (Deploy)
[+] Verified services on http://localhost:8000
[Pipeline] stage (Smoke Test)
[+] Health probe status: UP (Database: CONNECTED)
[Pipeline] End of Pipeline
Finished: SUCCESS
```

After the tests pass, the **Docker Build** stage checks that the Dockerfiles and the container configuration are valid, which corresponds to the `docker build` commands of Figure 5.7. The **Deploy** stage then confirms that the services are available on `http://localhost:8000`, the port published by the backend container in Figure 5.8. Finally, the **Smoke Test** stage calls the health endpoint and requires the answer "UP" with "Database: CONNECTED", the same answer that `curl` returned by hand in Figure 5.8. Only when all stages succeed does Jenkins print "Finished: SUCCESS"; any failing stage stops the pipeline and marks the build as failed.

The smoke test is the most important check of the deployment stage. The unit tests prove that the code is correct in isolation, but they cannot show that the deployed system is configured correctly: a wrong port, a missing environment variable or an unreachable database would pass every unit test and still fail for a real user. By calling the health endpoint of the running service, the smoke test proves that the deployed application starts and reaches the database.

The result of the deployment can be checked in three further places: `docker compose ps` shows both containers as healthy (Figure 5.8), Docker Desktop shows the container logs with the health checks answering 200 OK (Figure 5.9), and Nagios, described next, checks the services from outside. The deployment is repeatable because the whole system is described in files that are in Git: the `Jenkinsfile`, `docker-compose.yml` and the Dockerfiles.

5.6 Monitoring with Nagios (Experiment 7)

Once the stack is deployed, it has to be watched. The health checks of Docker (Figure 5.9) work from inside the containers and restart nothing by themselves; Nagios Core 4.5.12 works from outside, like a real user or an operator, and records the state of each service. The

configuration is kept in the folder `nagios-config` of the repository, with the active probes defined in commit `87c7874` (HMS-4) and the local Nagios server added in commit `c15c5d3`. Figure 5.11 is the Service Status Details page of Nagios.

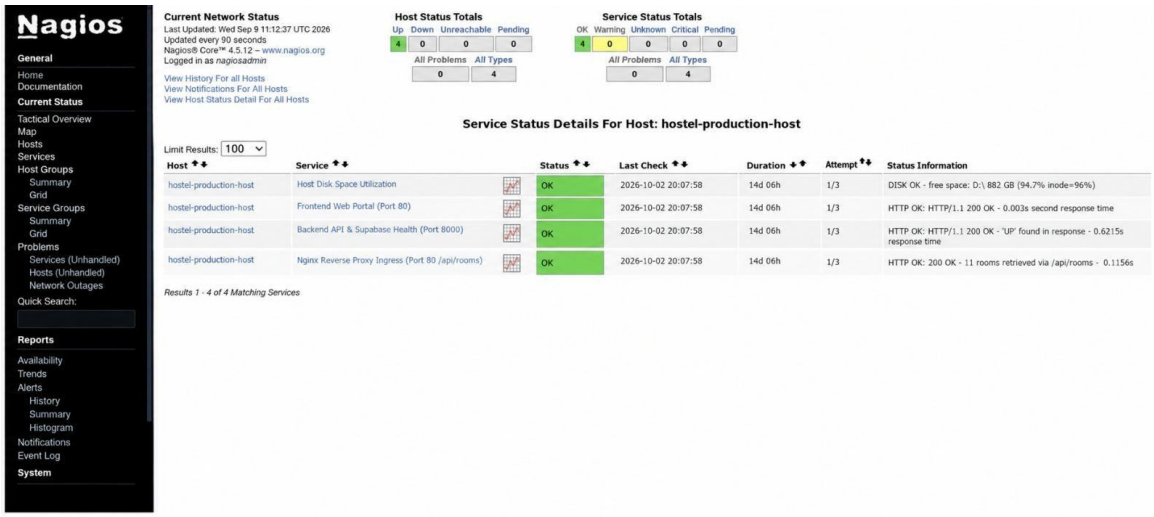


Figure 5.11: Nagios service status for host hostel-production-host

The page lists the services of the host **hostel-production-host**. At the top, “Host Status Totals” shows one host Up, and “Service Status Totals” shows four services OK, with nothing in Warning, Unknown, Critical or Pending, so the page confirms that the system was healthy at the time of the capture. The four checks are shown below.

Service	Check	Result shown
Host Disk Space Utilization	Free space on drive D: against the threshold	DISK OK, free space reported for D:
Frontend Web Portal (Port 80)	HTTP request to the Nginx container	HTTP OK: HTTP/1.1 200 OK, 0.003 s response time
Backend API & Supabase Health (Port 8000)	HTTP request to the health endpoint; the response must contain “UP”	HTTP OK: 200 OK, “UP” found in response, 0.621 s
Nginx Reverse Proxy Ingress (Port 80 /api/rooms)	HTTP request for the room list through the proxy	HTTP OK: 200 OK, 11 rooms retrieved via /api/rooms, 0.156 s

Table 5.5: Nagios services and their results

Each check tests something different. The frontend check shows that the Nginx container answers on port 80. The backend check goes further than “the port is open”: it fetches the health response and looks for the text “UP”, and because the health endpoint also tests the database connection (Section 4.5), a lost link to Supabase would turn this check red. The proxy check requests `/api/rooms` through port 80 and confirms that 11 rooms come back, which is the real business data and matches the “Total Rooms 11” on the dashboard (Figure 4.2); it therefore tests the complete chain browser, Nginx, FastAPI and database. The disk check watches the free space of drive D:, where the project and its test reports are stored, so that the pipeline does not fail silently because the disk is full.

The columns “Last Check”, “Duration” and “Attempt 1/3” show that the checks are repeated at regular intervals and that Nagios retries up to three times before it reports a hard failure, to avoid false alarms. Monitoring completes the loop of the pipeline: a problem found here would be raised again in Jira, fixed on a branch, and delivered through the same pull request and Jenkins run.

5.7 How CI/CD Is Achieved End to End

Continuous integration and continuous deployment in this project are not a single tool but a chain in which the output of each tool is the input of the next. The chain works as follows.

1. **Plan.** A story is created and assigned in Jira (Figures 2.1 to 2.4). The card in To Do is the request for change.
2. **Commit.** The team member works on a feature branch and commits with the stream prefix HMS-1 to HMS-4 (Figures 5.2 and 5.3).
3. **Integrate.** The branch is pushed to GitHub and merged into `main` through a pull request (Figures 5.4 and 5.6). Because every change joins the same branch, problems between the work of different members appear early and not at the end.
4. **Build and test (CI).** Jenkins checks out `main`, runs the 11 pytest tests and records the JUnit report (Figure 5.10). A failing test stops the pipeline.
5. **Package.** The Dockerfiles are checked and the backend and frontend images are built from the repository (Figure 5.7), so the application is delivered as the same artefact on every machine.
6. **Deploy (CD).** Docker Compose starts the two containers on `hostel-network` (Figure 5.8), and the Deploy stage confirms the services are reachable.
7. **Verify.** The smoke test calls `/health` and requires UP with the database CONNECTED (Figure 5.10); Docker health checks keep running (Figure 5.9).
8. **Monitor and close.** Nagios checks the frontend, the backend, the proxy path and the disk (Figure 5.11), and the story is moved to Done in Jira (Figures 2.5 to 2.8).

CI/CD stage	Tool	Input	Output	Evidence
Plan	Jira	Requirement	Story on the board	Figures 2.1 to 2.8
Source control	Git, GitHub	Code change	Commit, pull request, merge on main	Figures 5.2 to 5.6
Continuous integration	Jenkins, pytest	main branch	11 passed, junit.xml	Figure 5.10
Packaging	Docker	Dockerfiles and code	hostel-backend and hostel-frontend images	Figure 5.7
Continuous deployment	Docker Compose, Jenkins	Images	Running, healthy containers; smoke test UP	Figures 5.8, 5.9, 5.10
Monitoring	Nagios	Running services	Four services OK	Figure 5.11

Table 5.6: The CI/CD chain and the evidence for each stage

The practical benefit is speed with safety: after a merge, the same sequence of checks runs in the same order every time, so a change is either proven to work in the running system or stopped at the first failing stage. The remaining manual step in the demonstration run is the trigger ("Manual Trigger" in Figure 5.10); this is discussed with the limitations in Section 7.4.

6. Testing

6.1 Automated Tests

The backend is tested with pytest. The test file `backend/tests/test_main.py` contains eleven tests that exercise the operations of Table 3.2 and every branch of the allocation rules of Table 3.3. They run inside the Unit Tests stage of the Jenkins pipeline (Figure 5.10), and the results are written to `test-reports/junit.xml` in JUnit format so that Jenkins can record them.

No.	Test	What is verified
1	test_root_endpoint	The root address of the API answers
2	test_health_check_success	Health endpoint reports UP when the database is reachable
3	test_health_check_failure	Health endpoint reports a failure when the database is not reachable
4	test_list_rooms	The room list is returned with its fields
5	test_list_students	The student list is returned
6	test_allocate_student_success	A valid student is allocated to an available room
7	test_allocate_student_room_full	Allocation to a full room is rejected
8	test_allocate_student_room_maintenance	Allocation to a room under maintenance is rejected
9	test_allocate_student_room_not_found	Allocation to a room that does not exist is rejected
10	test_allocate_student_invalid_payload	A request with invalid data is rejected
11	test_dashboard_stats	Total rooms, occupied beds, vacancies and occupancy rate are computed correctly

Table 6.1: Automated test cases

Measure	Value
Test framework	pytest 9.1.1 (with anyio, langsmith and mock plugins) on Python 3.13.3
Tests collected	11
Tests passed	11 (100 %)
Duration	1.62 seconds
Report	test-reports/junit.xml (JUnit format)

Table 6.2: Result of the pytest run recorded by Jenkins

6.2 Deployment Smoke Tests

Unit tests prove the code; the smoke test proves the deployment. In the final stage of the pipeline the health endpoint of the running service is called, and the pipeline succeeds only when the answer is UP with the database CONNECTED (Figure 5.10). The same check was performed by hand with `curl` against port 8000 and against port 80 through Nginx (Figure 5.8), and both answered `{"status": "UP", "database": "CONNECTED"}`.

6.3 Functional Verification of the Interface

The figures in the dashboard were cross-checked against each other. The checks below can be repeated by anyone who has the screenshots of Chapter 4.

Check	Expected	Observed
Beds per room added together	2+2+2+2+1+1+1+3+3+3+2 = 22	22 total beds (Figure 4.2)
Occupied beds added together	2+1+1+0+1+1+0+2+3+1+0 = 12	12 occupied beds (Figure 4.2)

Check	Expected	Observed
Vacancies	22 − 12 = 10	10 available vacancies
Occupancy rate	12 ÷ 22 = 54.5 %	54.5 %
Students in registry	Equal to occupied beds	12 students (Figure 4.4)
Rooms offered in the form	Active rooms with an open bed: A-102, A-103, A-104, B-203, C-301, C-303	“6 Available” on the Assign Room list (Figure 4.3)
Room under maintenance	D-401 is not selectable	D-401 shows the Maintenance status
Rooms through the proxy	11 rooms	“11 rooms retrieved via /api/rooms” in Nagios (Figure 5.11)

Table 6.3: Cross-checks of the displayed data

6.4 Container and Monitoring Checks

- Docker health checks: both containers show the status healthy in `docker compose ps` (Figure 5.8), and the repeated `GET /health … 200 OK` lines in Docker Desktop show that the checks keep passing (Figure 5.9).
- Nagios: all four services are in the OK state and the host is Up (Figure 5.11).

7. Results and Discussion

7.1 Outcomes against Objectives

Objective	Outcome
Room inventory and filtering	Delivered: 11 rooms with occupancy bars, search and status filter (Figure 4.3)
Student allocation with validation	Delivered: form with client-side validation, backend rules tested by 5 tests (Figure 4.3, Table 6.1)
Live key figures	Delivered: total rooms, occupied beds, vacancies and occupancy rate (Figure 4.2)
Agile planning in Jira	3 epics, 6 stories, 19 story points; HM Sprint 1 completed with 6 completed work items (Figures 2.1 to 2.8)
Git workflow	15 commits, 3 contributors, 4 branches, pull requests #1 and #2 merged and verified (Figures 5.3 to 5.6)
Containerised stack, one command	2 images and 1 network started by `docker compose up -d --build`; both containers healthy (Figures 5.7 and 5.8)
Jenkins pipeline	Checkout, 11 tests passed in 1.62 s, Docker build check, deploy check and smoke test: Finished SUCCESS (Figure 5.10)
Monitoring	Nagios: 1 host Up, 4 services OK (Figure 5.11)

Table 7.1: Results against objectives

7.2 The Feedback Loop in Practice

The project shows the practices working together. The work was split into three streams that could proceed in parallel: Janvi's frontend branch, Soham's Docker and monitoring branch and Ananya's backend, Jenkins and Nagios work on `main`. Because each stream lived on its own branch (Figure 5.3), nobody blocked the others, and the two side branches were merged back by pull requests #1 and #2 (Figure 5.6). The Jira board (Figures 2.1 to 2.4) showed the same progress from the planning side, with cards moving from To Do to Done while the commits reached the repository.

Once the code was together on `main`, the pipeline gave immediate evidence about the combined result: the tests passed, the services started, the health probe answered UP, and Nagios confirmed from outside that the frontend, the backend and the data path were working. If any of these had failed, the cause would have been traced to a recent commit through its HMS prefix, fixed on a branch and delivered through the same route. This loop from plan to monitoring is what the seven experiments were meant to teach.

7.3 Challenges and Design Decisions

Challenge	Resolution
Three members worked on separate parts at the same time	One branch per stream (frontend-dashboard, devops-ci-cd) and merge through pull requests
Work of all members was pushed to GitHub through one account	Git keeps the original author of each commit; GitHub shows "authored by … and committed by Ananya5182" and lists three contributors (Figures 5.4 and 5.5)
A running process is not necessarily a working service	Health endpoint that also tests the database; used by Docker, the smoke test and Nagios
Containers must start together with the right network and ports	One docker-compose.yml with the private network hostel-network and published ports 80 and 8000

Challenge	Resolution
Allocation rules must hold for any client	Rules enforced in the backend, with one test for every rejected case
A shared database in the cloud must be reachable from the containers	Connection settings passed through environment variables; the CONNECTED state is tested by the health check

Table 7.2: Challenges and how they were resolved

7.4 Limitations

- The pipeline run documented in Figure 5.10 was started by hand (“Manual Trigger”). Jenkins runs on a development machine, so it cannot receive webhooks from GitHub; a periodic repository scan or a hosted Jenkins server would start a build automatically after every merge.
- The tests run on the host with Python 3.13.3, while the backend image is based on Python 3.11. Running pytest inside the container would remove this difference.
- The Docker Build and Deploy stages of the demonstration pipeline verify the Docker configuration and the availability of the services; building and starting the containers was shown with the Docker commands of Figures 5.7 and 5.8. There is no automatic rollback to the previous version when a deployment fails.
- Eleven tests cover the main operations and the allocation rules, but code coverage was not measured.
- Jenkins, the application and Nagios share one machine, so a failure of that machine would stop both the system and the means of observing it.
- The application has no login screen, so every visitor can allocate rooms; role-based access is future work.

8. Conclusion and Future Scope

8.1 Conclusion

The Hostel Management System meets its functional goal: a warden can register a student, choose only from rooms that are active and have a free bed, and see at once the effect on the occupancy figures, the room inventory and the student registry. The backend enforces the allocation rules, and automated tests cover every accepted and rejected case. More importantly for this course, the project shows the seven lab experiments operating as one system rather than as separate exercises.

Planning the work as epics and stories in Jira gave each member a clear stream and a visible board, and the sprint report confirmed that all six stories (19 story points) were completed. Git and GitHub kept the three members' work apart while it was in progress and together afterwards, with pull requests #1 and #2 as the points of integration. Docker and Docker Compose turned the environment into files that are versioned with the code, so the stack that was tested was the stack that ran. The Jenkins pipeline checked out `main`, ran the 11 tests, verified the build, deployed the services and ended with a smoke test that required the database to be connected. Nagios then watched the frontend, the backend, the proxy path and the disk. The lesson that repeats through the project is that each practice becomes more valuable because of the one that follows it.

8.2 Future Scope

- Start the Jenkins pipeline automatically after every merge, by scanning the repository or by hosting Jenkins where GitHub webhooks can reach it.
- Run the tests inside the container and push the images to a registry, so that deployment pulls a tagged, tested image.
- Add automatic rollback to the last healthy version when the smoke test fails.
- Add login and role-based access for wardens and students, and let students view their own room.
- Add fee tracking and payment, room change and check-out, and notifications to students.
- Measure code coverage, add automated frontend tests, and send Nagios alerts to email or a chat channel.
- Deploy to a college server or a Kubernetes cluster with monitoring separated from the application host.

References

1. FastAPI documentation. https://fastapi.tiangolo.com
2. Supabase documentation. https://supabase.com/docs
3. Pro Git, Scott Chacon and Ben Straub. https://git-scm.com/book
4. GitHub documentation: pull requests. https://docs.github.com/en/pull-requests
5. Docker documentation. https://docs.docker.com
6. Docker Compose documentation. https://docs.docker.com/compose
7. Nginx documentation. https://nginx.org/en/docs
8. Jenkins User Documentation: Pipeline. https://www.jenkins.io/doc/book/pipeline
9. pytest documentation. https://docs.pytest.org
10. Atlassian Jira Software documentation. https://support.atlassian.com/jira-software-cloud
11. Nagios Core documentation. https://www.nagios.org/documentation
12. Agile Software Development & DevOps Lab manual, Thadomal Shahani Engineering College, 2026–2027.

Appendix A: Running the Project

A.1 Prerequisites

- Git, Docker Desktop (with Docker Compose), Python 3 with pytest for the unit tests.
- Database settings for the Supabase project in a `.env` file, created from the template `.env.example`.

A.2 Steps

```
git clone https://github.com/Ananya5182/hostelmanagement.git
cd hostelmanagement
copy .env.example .env          (then enter the database settings)
docker compose up -d --build
docker compose ps
curl http://localhost:8000/health
curl http://localhost/health
```

Open `http://localhost` in a browser to use the dashboard. The expected answer of both `curl` commands is `{"status": "UP", "database": "CONNECTED"}`. To run the tests, execute pytest on the folder `backend/tests` from the project root; the pipeline does the same and writes `test-reports/junit.xml`.

A.3 Addresses

Component	Address	Purpose
Web client	http://localhost	Dashboard, allocation form, room inventory, student registry
Backend health	http://localhost:8000/health	Service and database status
Rooms through the proxy	http://localhost/api/rooms	Room list used by Nagios
Jenkins	http://localhost:8080	Pipeline console

Table A.1: Addresses used by the project

A.4 Repository Layout

Path	Content
backend/	FastAPI application and tests (backend/tests/test_main.py)
frontend/	Static web client, nginx.conf and Dockerfile
nagios-config/	Nagios host and service definitions
scripts/	Helper and verification scripts
docs/screenshots/	Screenshots used in documentation
Jenkinsfile, jenkins_server.py	Pipeline definition and the local Jenkins server
docker-compose.yml, docker-compose.monitoring.yml	Application stack and monitoring stack
database_schema.sql	Database schema
.env.example, .gitignore, README.md	Settings template, ignore rules, project description

Table A.2: Layout of the repository